Individual Study 2024 summer

08/28/24

Euiseok Han

Context

- How to download Mujoco?
- Constrained Policy Optimization paper review – again
  - Preliminary
  - Paper review
- Code review
- Future work
- Results
- Some appendix

How to download Mujoco?

How to download Mujoco?

- A stupid person buy a Mac even though his major is electrical engineering…

- To simulate CPO

1. Install mujoco engine
2. Install safety-gym and safety-starter-agent

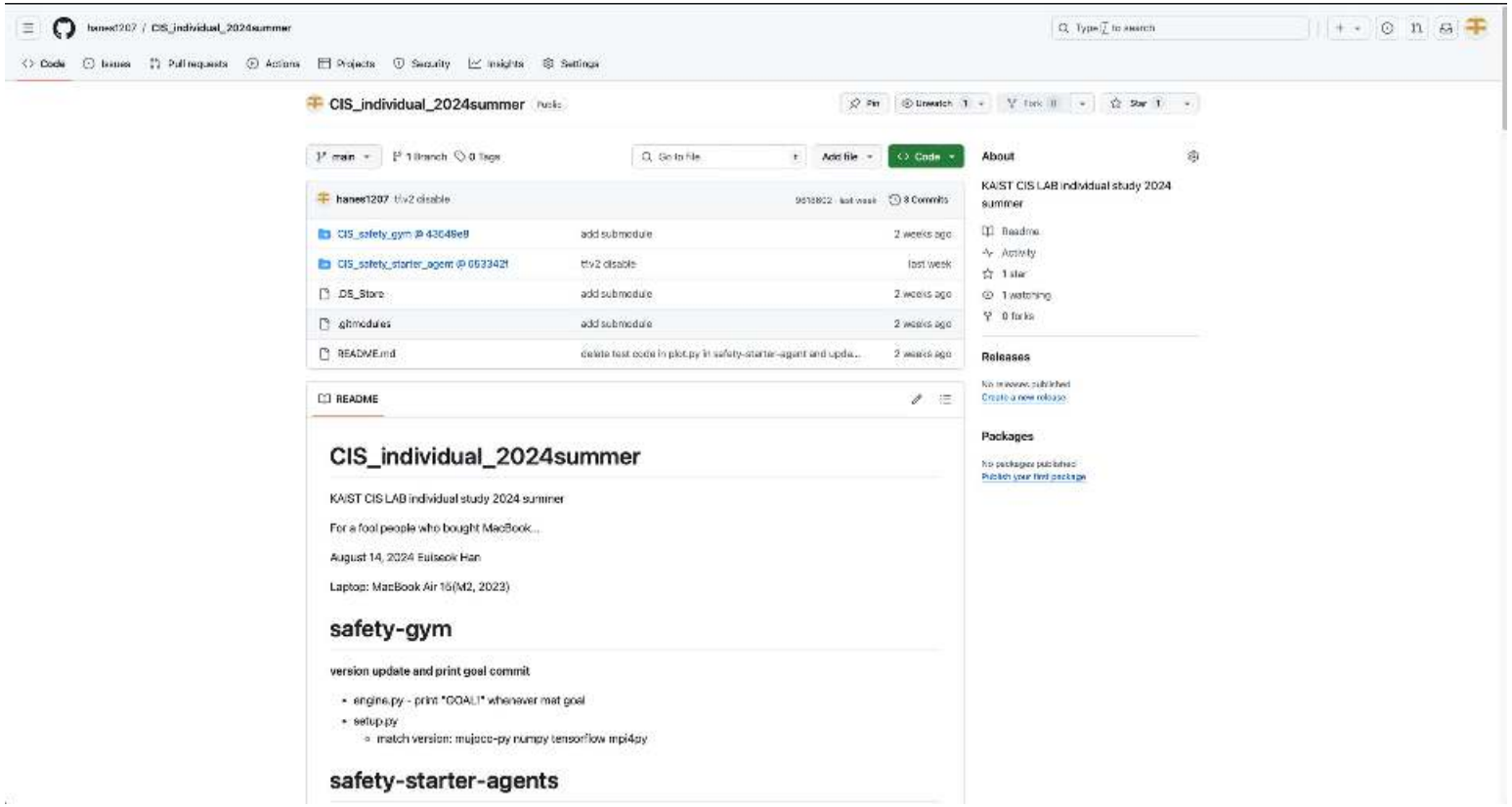
Problems:

1. While upgrading mujoco .v2.1.0 to .2.1.1, there were some errors
2. Difference between window and mac
3. Link errors
4. Others

How to download Mujoco?

- Fixed all and documented with gitlab!

https://github.com/hanes1207/CIS_individual_2024summer



How to download Mujoco?

safety-gym

version update and print goal commit

- engine.py - print "GOAL!" whenever met goal
- setup.py
  - match version: mujoco-py numpy tensorflow mpi4py

safety-starter-agents

version update and tf compatibility btw tf.v1 and tf.v2

- setup.py
  - match version: mpi4py mujoco-py numpy tensorflow
- network.py
  - tf compatibility btw tf.v1 and tf.v2
    - import tensorflow as tf --> import tensorflow.compat.v1 as tf
    - ++ tf.disable_eager_execution()
- run_agent.py, trust_region.py, sac.py, load_utils.py, logx.py, mpi_tf.py
  - tf compatibility btw tf.v1 and tf.v2
- plot.py
  - ++ import pdb # for debugging
  - matplotlib no longer has tsplot function: sns.tsplot --> sns.lineplot
  - set ylim: plt.ylim(bottom=0) / plt.ylim(top=30)
- comparision.png
  - result sample

What's different

How to run download mujoco?

- Reference sites
  - https://ropiens.tistory.com/178?category=1027022
  - openai/mujoco-py#662
  - https://github.com/google-deepmind/mujoco/releases/tag/2.1.1

본인은 **https://github.com/google-deepmind/mujoco/releases/download/2.1.1/mujoco-2.1.1-macos-universal2.dmg** 파일을 **~/Volumns/MuJoCo안의 파일들을 ~/mujoco/안에 넣어놓음**

1. Make Virtual Environment

- use mini-forge, not anaconda: brew install mini-forge
- conda create -n safetym python=3.9.19
- when deactivating: conda deactivate

2. Download mujoco engine

- mkdir -p $HOME/.mujoco/mujoco210
- cd ~/.mujoco/mujoco210
- mkdir bin include
- cd ~/.mujoco/mujoco210/bin
- cp -r ~/mujoco/MuJoCo.framework/Versions/Current/libmujoco.2.1.1.dylib libmujoco210.dylib
- cd ~/.mujoco/mujoco210/include
- cp -r ~/mujoco/MuJoCo.framework/Versions/Current/Headers/ .
- brew install glfw
- cd ~/.mujoco/mujoco210/bin
- ln -s /opt/homebrew/lib/libglfw.3.dylib .
- export CC=/opt/homebrew/bin/gcc-11

3. install mujoco_py, safety-gym, safety-starter-agent

- git clone https://github.com/openai/safety-gym
- git clone https://github.com/openai/safety-starter-agents
- cd ~/safety-gym
- pip3 install -e .
- cd ~/safety-starter-agents
- pip3 install -e .

4. import mujoco_py

- python
- import mujoco_py

Detailed instruction

How to download Mujoco?

Troble Shooting 2

```
(safegym) hanes1207@EuiseokHan-MacbookAir safety-starter-agents % python
Python 3.9.19 | packaged by conda-forge | (main, Mar 20 2024, 12:55:20)
[Clang 16.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import mujoco_py
Compiling /opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/python3.9/site-packages/mu
[1/1] Cythonizing /opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/python3.9/site-pack
ld: warning: dylib (/opt/homebrew/Cellar/gcc@11/11.4.0/lib/gcc/11/libgomp.dylib) was built f
ld: warning: dylib (/Users/hanes1207/.mujoco/mujoco210/bin/libglfw.3.dylib) was built for ne
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/python3.9/site-packages/mujoc
    from mujoco_py.builder import cymj, ignore_mujoco_warnings, functions, MujocoException
  File "/opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/python3.9/site-packages/mujoc
    cymj = load_cython_ext(mujoco_path)
  File "/opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/python3.9/site-packages/mujoc
    mod = load_dynamic_ext('cymj', cext_so_path)
  File "/opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/python3.9/site-packages/mujoc
    return loader.load_module()
ImportError: dlopen(/opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/python3.9/site-pa
  Referenced from: <98B601EE-8E50-32AF-924D-D6F6A7B2A8AE> /opt/homebrew/Caskroom/miniforge/b
  Reason: tried: '/opt/homebrew/Caskroom/miniforge/base/envs/safegym/lib/MuJoCo.framework/Ve
```

export DYLD_LIBRARY_PATH=$DYLD_LIBRARY_PATH:~/mujoco/MuJoCo.app/Contents/Frameworks/MuJoCo.framework/Versions/Current

it works!

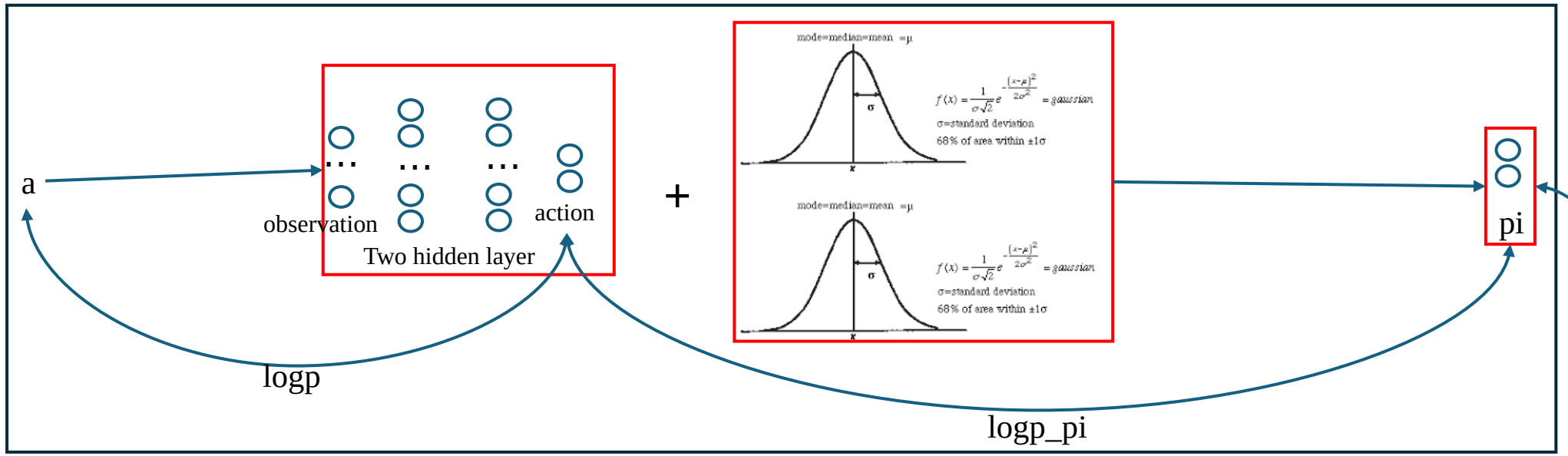
5. Plotting graph

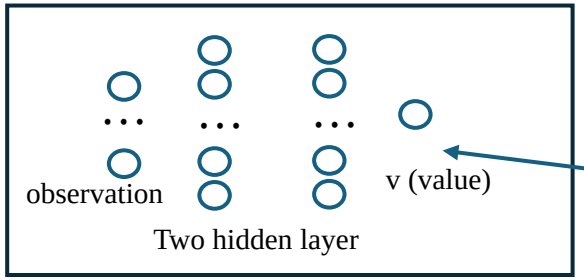
- python plot.py ../data

Some techniques for troble shooting

Constrained Policy Optimization paper review - again

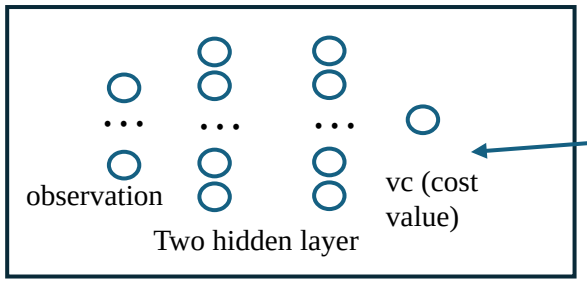
Constrained Policy Optimization paper review - again





ret

train



cret

train

train

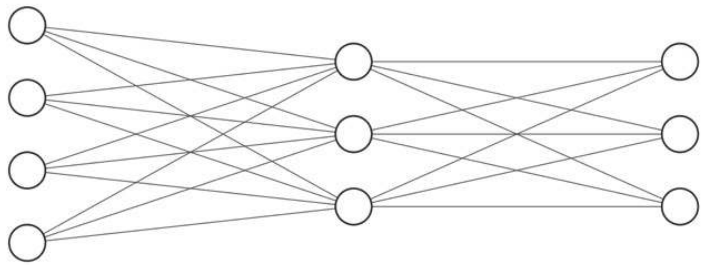
pi_loss
v_loss
vc_loss

Calculated at last local steps of epoch
by interacting with env

Some background knowledges…

1. Neural network
2. Multilayer Perceptron
3. Loss functions (Cross Entropy, KL-divergence…)
4. How to train policy? (Back propagation)
5. Dual problem

Neural network



해당 층은 노드(node)가 3개다. 이미 익숙한 주어진 코드와 같은 형식으로 코드를 쓰도록 하겠다. 추가된 층에 맞춰서 출력 값 이름도 바꿔주도록 한다.

```
입력값 = [[2.3, 5.6, 1.2, 2.4],
          [1.0, 2.0, 3.0, 2.5],
          [2.0, 5.0, -1.0, 2.0]]

가중치 = [[3.1, -2.1, 8.7, -9.1],
          [4.5, 2.1, -1.7, 3.4],
          [2.3, -6.4, 2.1, -6.3]]

편향 = [2.0, 3.0, 1.0]

가중치2 = [[0.1, -0.14, 0.5],
           [-0.5, 0.12, -0.33],
           [-0.44, 0.73, -0.13]]

편향2 = [-1.0, 2.0, -0.5]

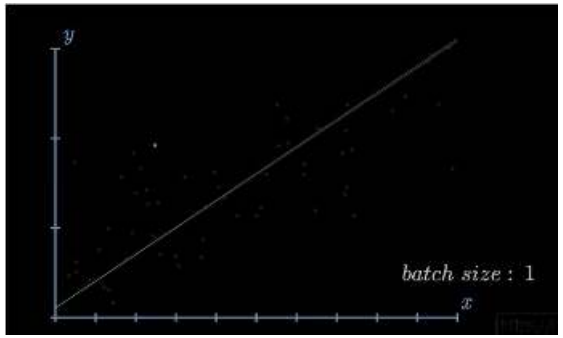
1층출력값 = np.dot(입력값, np.array(가중치).T) + 편향

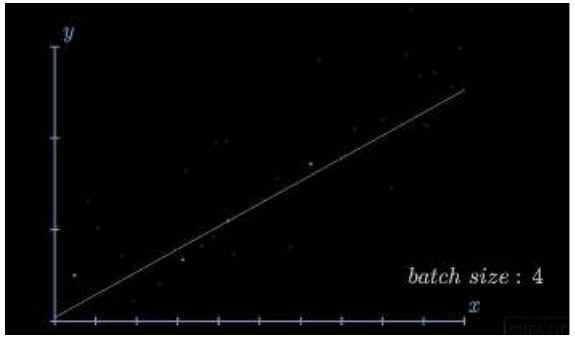
2층출력값 = np.dot(1층출력값, np.array(가중치2).T) + 편향2
print(2층출력값)
```

What is

우선의 배치(batch)의 개념부터 다뤄보도록 하겠다. 우선 배치 학습을 통한 모델 학습은 데이터를 몇 개로 나눠서 단순한 작업을 여러 코어에 나눠 효율적인 병렬 연산(parallel computing)을 수행하고자 하는 목적도 존재한다. 그러나 배치를 사용하는 또 다른 목적은 우리가 훈련시키고 있는 모델이 (그러니까 지금 우리가 만들고 있는 인공신경망이) 아직 관측한 적이 없는 데이터에 대한 올바른 예측을 수행하도록 도와주는 일반화(generalization) 작업이기도 하다.

반면 점을 한번에 여러 개씩 주면, 여러분은 선형 회귀 모델의 선이 데이터 전체에 대해 훨씬 정확하게 오차를 줄여나가는 과정을 겪는 것을 볼 수 있을 것이다. 여기서 의문점이 생길지도 모른다. 한 번에 데이터를 다 보여주면 안 될까? 안타깝게도 우리가 실제로 처리해야 하는 데이터는 몇 만개 이상일 수도 있다. 이를 한 번에 다 써서 모델을 훈련시켜버리면 굉장히 오래 걸리고 비효율적인 계산 과정이 일어난다. 특히나 컴퓨터의 연산 처리 능력이 한정된 현실의 상황에서는 배치는 필수적으로 사용해야하는 기법이다. 여기에 더해 모델이 학습 데이터에 과하게 맞아버리는 과적합(overfitting) 현상이 발생하여 표본 외의 데이터 상대로 예측 능력이 떨어지는 모델이 만들어져 오히려 우리가 추구하고자 했던 일반화의 실패가 발생한다.





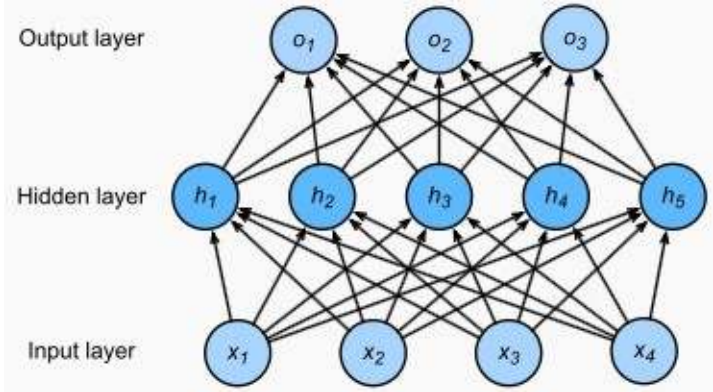
Multilayer Perceptron

다중 클래스 분류의 경우 위 그림이 수학적으로 어떻게 정의되는지 보겠습니다.

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$$
$$\mathbf{o} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o})$$

위 방법의 문제점은 은닉층을 $\mathbf{W} = \mathbf{W}_2\mathbf{W}_1$ 과 $\mathbf{b} = \mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2$ 를 사용해서 단일층 퍼셉트론(single layer perceptron) 식으로 재구성할 수 있기 때문에, 다층(mutilayer)이 아닌 단순한 단일층 퍼셉트론(single layer perceptron)이라는 점입니다.

$$\mathbf{o} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2 = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}\mathbf{x} + \mathbf{b}$$



이를 해결하는 방법은 모든 층의 다음에 $\max(x, 0)$ 와 같은 비선형 함수 σ 를 추가하는 것입니다. 이렇게 하면, 층들을 합치는 것이 더 이상 불가능해집니다. 즉,

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$$
$$\mathbf{o} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o})$$

→

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$\mathbf{o} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o})$$

3.8.5. 활성화 함수(activation function)

활성화 함수(activation function)의 예를 더 알아보겠습니다. 결국 딥 네트워트를 동작시키는 것은 선형(linear) 항목과 비선형(nonlinear) 항목들을 서로 교차시키는 것입니다. 구현하기 간단하고 좋은 효과로 유명한 ReLU 함수가 있습니다.

3.8.6. ReLU 함수

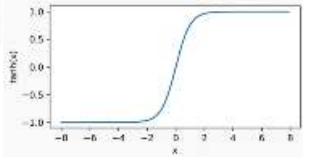
ReLU (rectified linear unit) 함수는 아주 간단한 비선형 변환입니다. 주어진 x 에 대해서, ReLU 함수는 다음과 같이 정의됩니다.

$$\text{ReLU}(x) = \max(x, 0).$$

3.8.7. Sigmoid 함수

sigmoid 함수는 실수 값을 (0,1) 사이의 값으로 변환해줍니다.

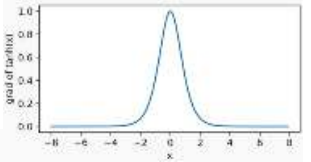
$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$



3.8.8. Tanh 함수

Tanh (Hyperbolic Tangent) 함수는 값을 -1와 1사이 값으로 변환합니다.

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$



3.8.4. 벡터화와 미니 배치

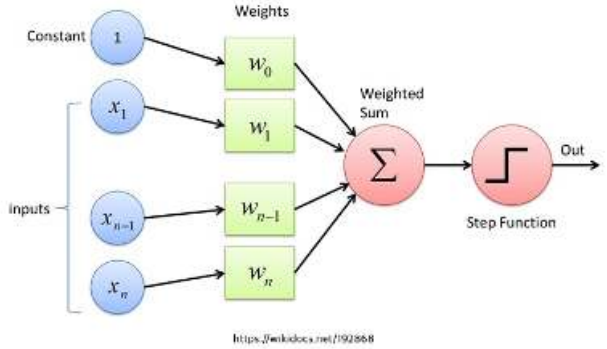
샘플들이 미니 배치로 주어지는 경우에는 벡터화를 통해서 구현의 효율성을 높일 수 있습니다. 요약하면, 벡터를 행렬로 바꿀 것입니다. $\mathbf{X}$ 는 미니 배치의 입력 행렬에 대한 표기입니다. 2개의 은닉층(hidden layer)를 갖는 MLP는 다음과 같이 표현됩니다.

$$\mathbf{H}_1 = \sigma(\mathbf{W}_1\mathbf{X} + \mathbf{b}_1)$$
$$\mathbf{H}_2 = \sigma(\mathbf{W}_2\mathbf{H}_1 + \mathbf{b}_2)$$
$$\mathbf{O} = \text{softmax}(\mathbf{W}_3\mathbf{H}_2 + \mathbf{b}_3)$$

https://ko.d2l.ai/chapter_deep-learning-basics/mlp.html

Multilayer Perceptron

퍼셉트론 (Perceptron)



단층 퍼셉트론이라고 불리는 인공신경망의 가장 기본적인 형태 중 하나로, 입력과 출력 사이에 하나의 계층이 있는 모델.

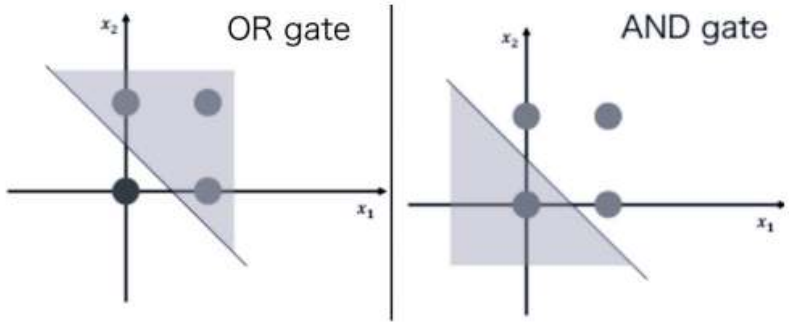
구조

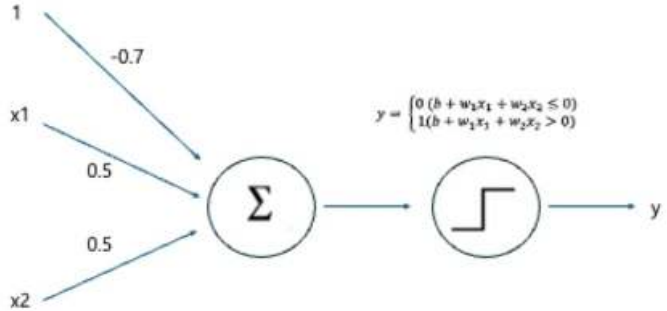
- **입력 (Input)**: 단층 퍼셉트론의 입력은 벡터로 표현, 이 벡터는 입력 특성(Feature)을 나타낸다.
  - **입력층의 노드 수는 Feature의 수와 일치**한다.
  - $xT = (x1, x2, \cdots xn)$
- **가중치 (Weight)**: 각 입력 특성에 대해 가중치가 할당. 가중치는 모델이 학습하는 매개변수로, 특징의 중요도를 나타낸다.
  - $wT = (w1, w2, \cdots wn)$
  - $w0 = b$ (bias(편향값)), 활성화 함수의 결정 경계를 조정하여 모델이 더 유연하게 데이터를 학습할 수 있도록 역할을 한다.
- **가중합 (Weighted Sum)**: 입력과 가중치의 곱을 모두 합한 값
  - $\sum_{i=1}^{N} w_i x_i + w0$ => $w1 \cdot x1 + w2 \cdot x2 + w3 \cdot x3 \ldots + wn \cdot xn + w0 \cdot 1$
- **활성함수 (Activation Function)**: 가중합을 입력으로 받아 출력을 생성하는 함수, 단층 퍼셉트론에서는 **계단 함수** 사용
  - $g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{else} \end{cases}$
- **출력**: 입력과 가중치의 선형 조합을 계산하고, 그 결과에 대해 활성함수를 적용하여 출력을 생성 (가중합 + 활성 함수)
  - **출력층의 노드 수는 예측하고자 하는 타겟 값(Label)의 수와 일치**한다.
  - $f(x) = f(wTx + w0)$

퍼셉트론의 활용 예시

단층 퍼셉트론은 선형 결정 경계를 사용한다. 따라서 선형 분류의 용도로 사용이 가능하다.

가장 대표적인 예시로 AND Gate와 OR Gate의 구현이 있다. 두 개의 논리회로는 대표적으로 선형 분류가 가능한 친구들이다.



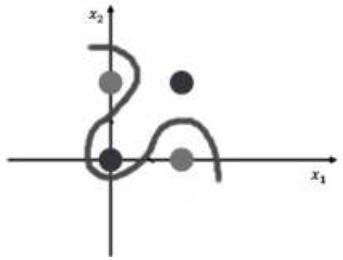


$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

$0.5 * 0 + 0.5 * 0 - 0.7 = -0.7 \rightarrow 0$
$0.5 * 0 + 0.5 * 1 - 0.7 = -0.2 \rightarrow 0$
$0.5 * 1 + 0.5 * 0 - 0.7 = -0.2 \rightarrow 0$
$0.5 * 1 + 0.5 * 1 - 0.7 = 0.3 \rightarrow 1$

한계

단층 퍼셉트론은 비선형적인 결정 경계를 필요로 하는 문제에는 성능이 제한되는 한계가 있다. 가장 대표적인 문제의 사례가 XOR Gate의 구현이다.

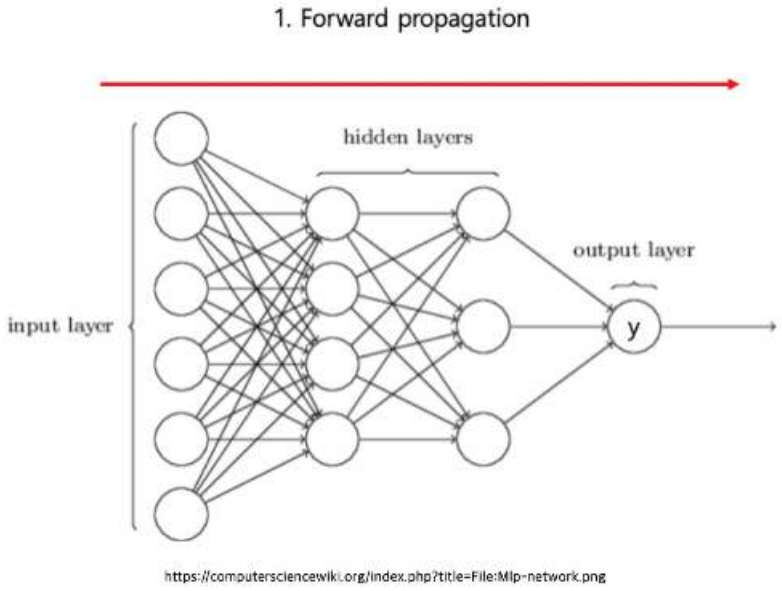


Multilayer Perceptron

다층 퍼셉트론은 학습 알고리즘을 사용하여 모델을 훈련한다. 앞에서 말했듯 **모델을 훈련한다는 것은 적절한 가중치를 찾아가는 과정**이다.
이때 사용되는 손실 함수는 모델의 예측과 실제 값 간의 차이를 측정. 역전파(backpropagation) 알고리즘이 사용, 경사하강법과 같은 최적화 알고리즘을 통해 손실 함수를 최소화한다.

1. Forward Propagation (순전파)

우리는 앞에서 살펴보았듯 입력층에서부터 여러 은닉층을 거쳐 ☒**예측 결과인 출력을 내는 과정**을 살펴보았었다. 해당 과정을 **Forward Propagation**이라고 한다.

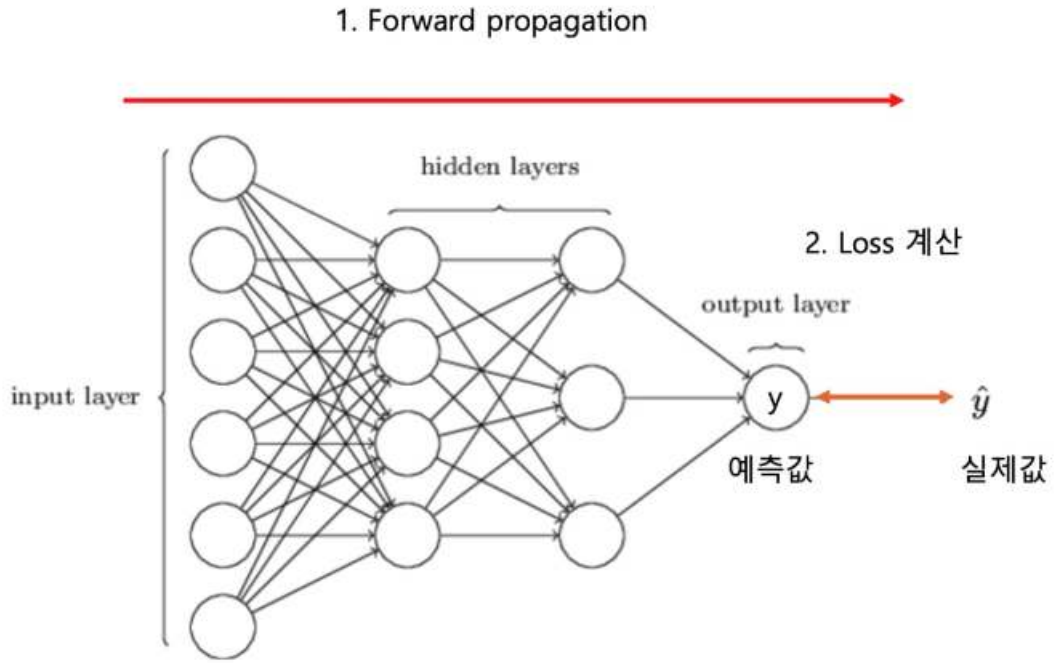


https://computersciencewiki.org/index.php?title=File:Mlp-network.png

- **입력층**: 훈련 데이터(Feature)의 입력
- **가중치 및 활성화 함수 적용**: 각 은닉층과 출력층은 가중합을 계산, 그 결과에 활성화 함수를 적용. 해당 과정을 입력층에서부터 출력층까지 반복한다.
- **출력층**: 모델의 예측 결과를 얻어낸다.

2. Loss 계산

Loss는 예측한 결과와 실제 결과간의 차이를 나타내주는 지표이다. 기계학습의 목표는 학습을 통해서 Loss를 최소화하는 것이다.
실제값과 모델의 예측값 사이의 차이를 나타내는 손실 함수(MSE, ☒Cross Entropy Loss 등)를 사용하여 계산한다.



Loss functions

In mathematical optimization and decision theory, a **loss function** or **cost function** (sometimes also called an error function)[1] is a function that maps an event or values of one or more variables onto a real number intuitively representing some "cost" associated with the event. An optimization problem seeks to minimize a loss function. An **objective function** is either a loss function or its opposite (in

- 평균 제곱 오차 (Mean Squre Error, MSE)

$$\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$$

(Mean Square Error, MSE)

- 평균 제곱근 오차 (Root Mean square Error, RMSE)

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}$$

(Root Mean Squre Error, RMSE)

- 이진 교차 엔트로피 (Binary Cross-entropy)

$$-\frac{1}{n}\sum_{i=1}^{n}(y_i log\hat{y}_i+(1-y_i)log(1-\hat{y}_i))$$

(Binary Cross-entropy)

- 카테고리컬 교차 엔트로피 (Categorical Cross-entropy)

$$-\sum_{i=1}^{n}y_i log(a_i)$$

(Categorical Cross-entropy)

(여기서 \(y_i\)는 타깃, \(\hat{y}_i \)는 모델에 값을 대입했을 때 나오는 예측값입니다.)

https://yhyun225.tistory.com/5
https://en.wikipedia.org/wiki/Loss_function

Loss functions

Claude Shannon 의 Entropy

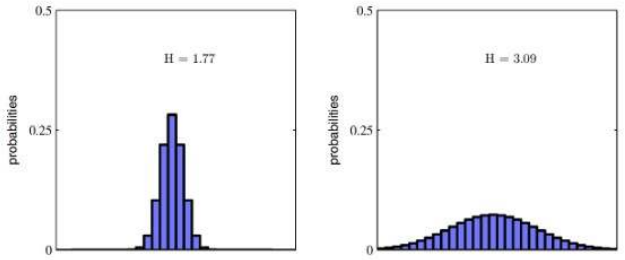
$$H = \sum_{i} (\text{사건 발생확률}) \cdot \log_2\left(\frac{1}{\text{사건 발생확률}}\right)$$

$$= \sum_{i} p_i \log_2\left(\frac{1}{p_i}\right)$$

$$= -\sum_{i} p_i \log_2(p_i)$$

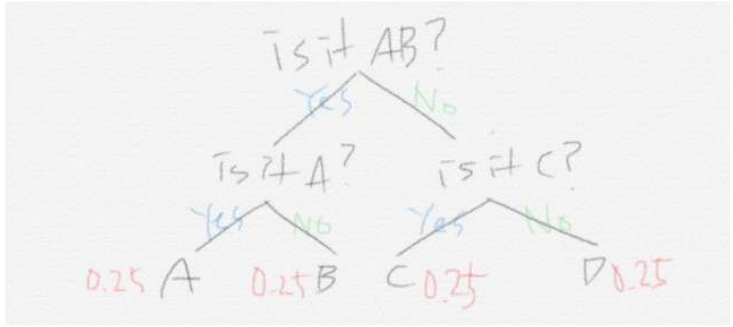
질문 횟수

Entropy는 가능한 모든 사건이 같은 확률로 일어날 때 그 최댓값을 갖습니다. 그 각각의 확률이 모두 동등한 상황에서 조금만 벗어나도 entropy는 감소합니다. 아래 두 경우를 생각해보겠습니다. X의 범위에 걸쳐 특정 값들만 높은 왼쪽 분포가 값들끼리 서로 비슷비슷한 오른쪽 분포보다 entropy가 작습니다.

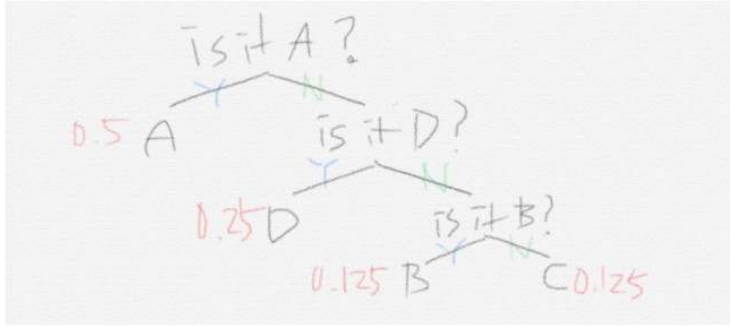


https://hyunw.kim/blog/2017/10/14/Entropy.html

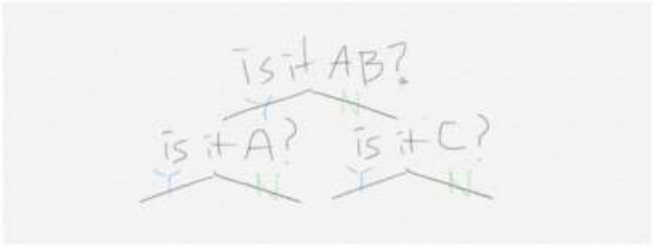
Loss functions



Machine Q



Machine P



Strategy Q

이 **전략 Q**를 **기계 P**에 적용해보겠습니다. 그렇다면 앞선 식에서 질문의 개수가 바뀌고 이에 따라 계산된 entropy도 아래처럼 바뀝니다.

$$0.5 \times 2 + (0.125 \times 2) \times 2 + 0.25 \times 2 = 2$$

확률분포로 된 어떤 문제 p에 대해 확률분포로 된 어떤 전략 q를 사용할 때의 질문개수의 기댓값이 바로 cross entropy입니다.

Cross entropy의 다양한 쓰임새

$$\begin{aligned} H(p,q) &= \sum_i p_i \log_2 \frac{1}{q_i} \\ &= -\sum_i p_i \log_2 q_i \end{aligned}$$

https://hyunw.kim/blog/2017/10/14/Entropy.html

Loss functions

$$H(p,q) = \sum_i p_i \log_2 \frac{1}{q_i}$$
$$= -\sum_i p_i \log_2 q_i$$

Cross entropy를 수식으로 적어보면 이렇습니다. 대개 머신러닝에서 cross entropy를 사용할 때는 이렇습니다: p_i가 특정 확률에 대한 참값 또는 목표 확률이고, q_i가 우리가 현재 학습한 확률값입니다. 예를 들어 여기서는 $p = [0.5,\ 0.125,\ 0.125,\ 0.25]$ 이고, $q = [0.25,\ 0.25,\ 0.25,\ 0.25]$ 가 되는 셈입니다. 따라서, 우리가 어떤 q_i 를 학습하고

Weight 를 학습시켜 나가자

연속형 확률분포에서는 $-\int p(x) \log q(x) dx$

Binary classification과 cross entropy

$$p = [\, y,\ 1-y \,]$$
$$q = \left[\, \hat{y},\ 1-\hat{y} \right]$$

$$-y \log \hat{y} - (1-y) \log(1-\hat{y})$$

Cross entropy는 log loss로 불리기도 합니다. 왜냐하면 cross entropy를 최소화하는 것은 log likelihood를 최대화하는 것과 같기 때문입니다. 찬찬히 살펴보겠습니다. 어떤 데이터가 0 또는 1로 predict될 확률은 $\hat{y}, 1-\hat{y}$이므로 그 데이터의 likelihood 식을 이렇게 세워볼 수 있습니다. (y가 0 또는 1의 값만 가질 때 가능합니다)

$$\hat{y}^{y} (1-\hat{y})^{(1-y)}$$

$y = 1$일 때 $\hat{y}$를 최대화시켜야 하고, $y = 0$일 때는 $(1-\hat{y})$를 최대화시켜야 합니다. 여기에 log를 씌워보겠습니다. 그러면 지수들이 내려와서 이렇게 됩니다:

$$maximize\ \ y \log \hat{y} + (1-y) \log(1-\hat{y})$$
$$= minimize\ \ -y \log \hat{y} - (1-y) \log(1-\hat{y})$$

https://hyunw.kim/blog/2017/10/14/Entropy.html

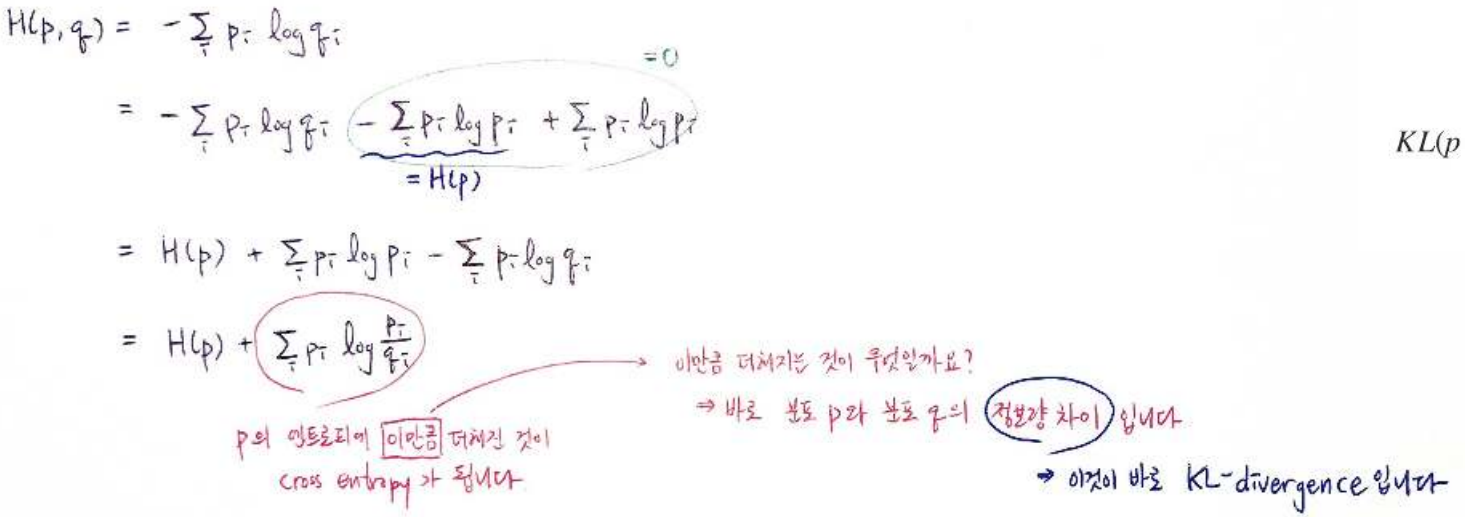
Loss functions

> **쿨백-라이블러 발산**(Kullback-Leibler divergence, **KLD**)은 두 확률분포의 차이를 계산하는 데에 사용하는 함수로, 어떤 이상적인 분포에 대해, 그 분포를 근사하는 다른 분포를 사용해 샘플링을 한다면 발생할 수 있는 정보 엔트로피 차이를 계산한다. 상대 엔트로피(relative entropy), 정보 획득량(information gain), 인포메이션 다이버전스(information divergence)라고도 한다.

$$KL(p\|q) = H(p,q) - H(p)$$

KL-divergence의 정확한 식은 이렇습니다. 대개 $D_{KL}(p|q)$ 또는 $KL(p|q)$로 표현합니다.

$$KL(p \parallel q) = \begin{cases} \sum_i p_i \log \frac{p_i}{q_i} \text{ 또는 } -\sum_i p_i \log \frac{q_i}{p_i} & \text{(이산형)} \\ \int p(x) \log \frac{p(x)}{q(x)} dx \text{ 또는 } -\int p(x) \log \frac{q(x)}{p(x)} dx & \text{(연속형)} \end{cases}$$



들어있습니다. 그 $H(p)$에 무언가 더해진 것이 cross entropy입니다. 이때 이 무언가 더해지는 것이 바로 "정보량 차이"인데, 이 정보량 차이가 바로 KL-divergence입니다.

KL-divergence의 특성

KL-divergence의 몇 가지 특징들이 있는데, 대표적인 것이 2가지입니다.

- $KL(p|q) \geq 0$
- $KL(p|q) \neq KL(q|p)$
  - "KL-divergence는 거리 개념이 아니다"라는 말로 매우 자주 등장합니다

https://hyunw.kim/blog/2017/10/14/Entropy.html

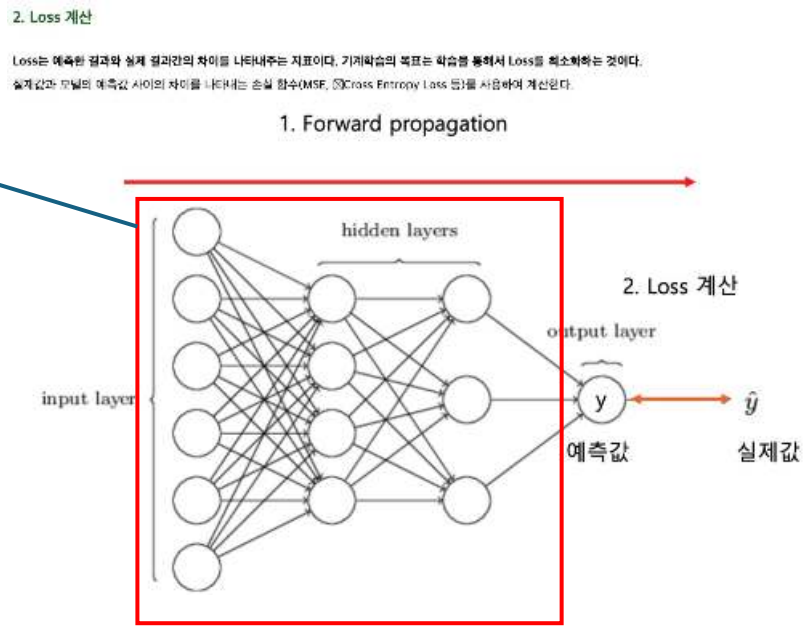
Loss functions

KL-divergence와 log likelihood

우리가 전체를 알 수 없는 분포 $p(x)$ 에서 추출되는 데이터를 우리가 모델링하고 싶다고 가정해보겠습니다. 우리는 이 분포에 대해 어떤 학습 가능한 parameter θ의 parametric distribution $q(x|\theta)$ 를 이용해 근사시킨다고 가정해보겠습니다. 이 θ 를 결정하는 방법 중 하나는 바로 $p(x)$와 $q(x|\theta)$ 사이의 KL-divergence 를 최소화시키는 θ 를 찾는 것입니다. 우리가 $p(x)$ 자체를 모르기 때문에 이는 직접 해내기는 불가능합니다. 하지만 우리는 이 $p(x)$에서 추출된 몇 개의 샘플 데이터(training set)는 압니다(x_n, for $n = 1, \dots, N$). 따라서 $p(x)$에 대한 기댓값은 그 샘플들의 평균을 통해 구할 수 있습니다.

$$KL(p\|q) \simeq \frac{1}{N}\sum_{n=1}^{N}\{-\ln q(x_n|\theta) + \ln p(x_n)\}$$

$\ln p(x_n)$ 는 θ에 대해 독립이고, $-\ln q(x_n|\theta)$ 는 training set으로 얻은 $q(X|\theta)$ 분포 하에서 θ 에 대한 negative log likelihood 입니다. 그러므로 KL-divergence를 minimize하는 것이 likelihood를 maximize하는 것과 같다는 것을 알 수 있습니다. [Pattern Recognition and Machine Learning, C.M. Bishop 참조]

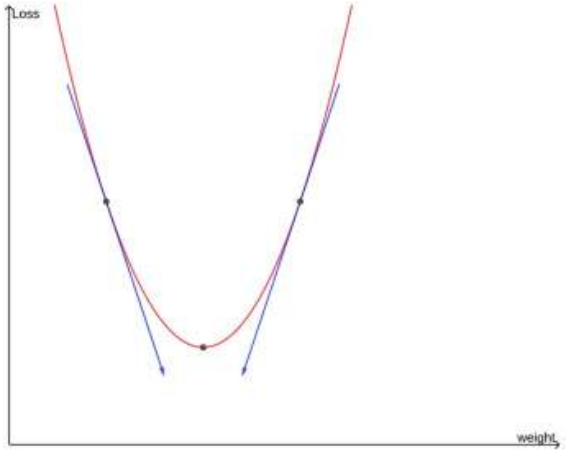


https://hyunw.kim/blog/2017/10/14/Entropy.html

MLP – How to train?

가중치에 대한 차원

경사하강법은 기울기를 이용하는 방법이라고 했습니다. 두 점의 기울기를 살펴보겠습니다.



규칙이 보이네요. w값에서의 손실 함수의 미분계수가 음이라면 w를 양의 방향으로, 손실 함수의 미분계수가 양이라면 w를 음의 방향으로 이동시켜 주어야 합니다.

이제 w를 얼마만큼 이동시키는지가 관건이겠네요. 경사하강법에서는 다음과 같은 식으로 w값을 조정해줍니다.

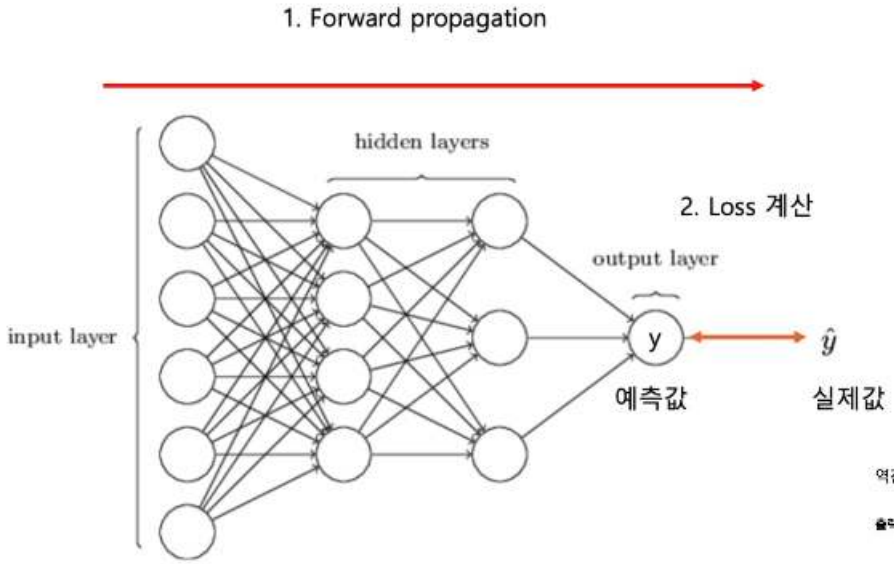
$$w = w - \alpha \times \frac{\partial L}{\partial w}$$

(w : weight, \(\alpha \) : learning-rate)

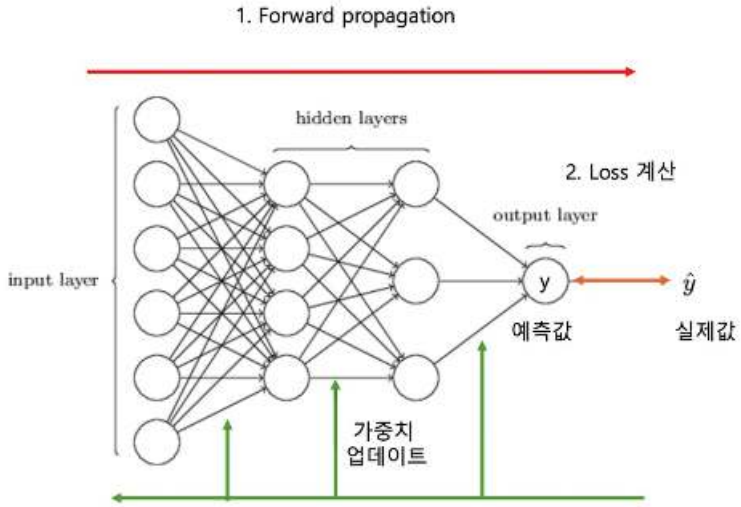
2. Loss 계산

Loss는 예측한 결과와 실제 결과간의 차이를 나타내주는 지표이다. 기계학습의 목표는 학습을 통해서 Loss를 최소화하는 것이다.

실제값과 모델의 예측값 사이의 차이를 나타내는 손실 함수(MSE, ☒Cross Entropy Loss 등)를 사용하여 계산한다.

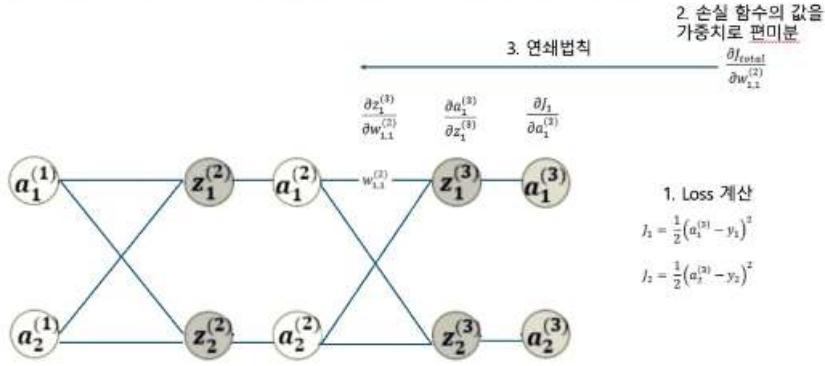


3. Backpropagation (역전파)



역전파 알고리즘

출력층에서 입력층으로 거꾸로 진행하면서 각각의 Hidden Layer의 각각의 가중치들을 업데이트 시켜주는 작업을 진행한다.



Dual problem

$$\theta_{k+1} = \arg\max_{\theta} \quad g^T(\theta - \theta_k)$$
$$\text{s.t.} \quad c_i + b_i^T(\theta - \theta_k) \leq 0 \quad i = 1, ..., m$$
$$\frac{1}{2}(\theta - \theta_k)^T H(\theta - \theta_k) \leq \delta. \tag{11}$$

↔

dual

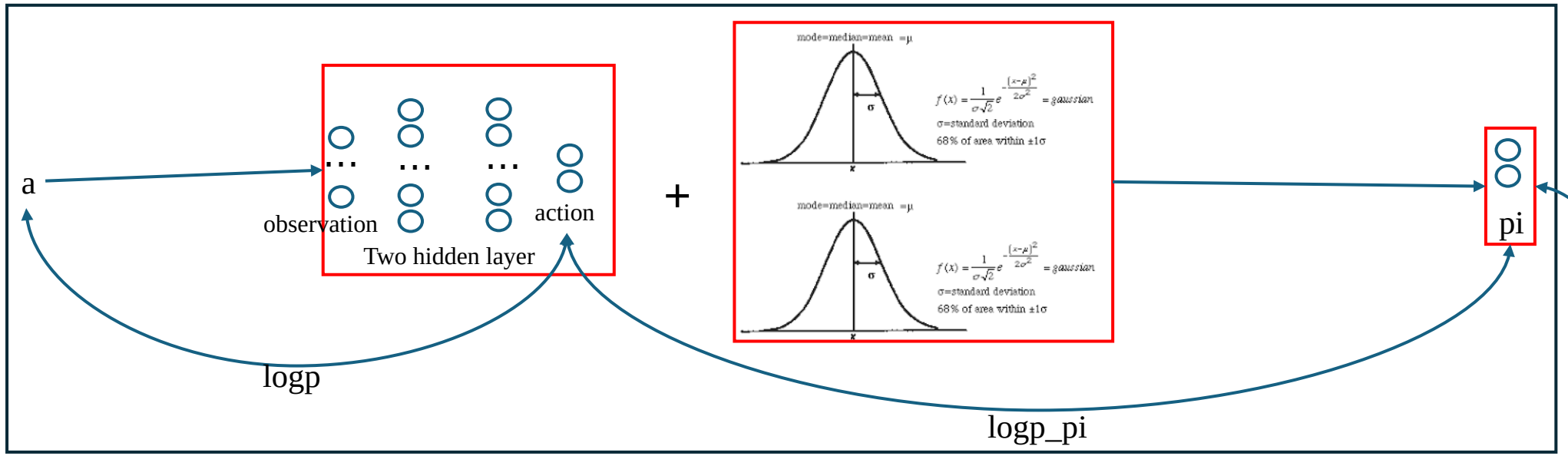
ble for the next subsection.) With $B \doteq [b_1, ..., b_m]$ and $c \doteq [c_1, ..., c_m]^T$, a dual to (11) can be expressed as

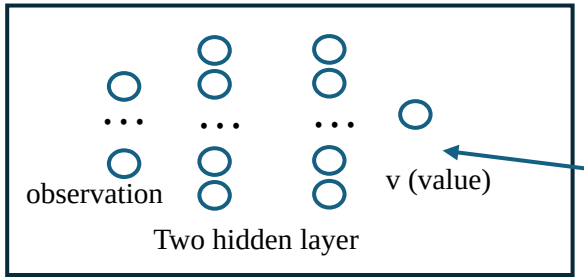
$$\max_{\substack{\lambda \geq 0 \\ \nu \succeq 0}} \frac{-1}{2\lambda}\left(g^T H^{-1} g - 2r^T \nu + \nu^T S \nu\right) + \nu^T c - \frac{\lambda\delta}{2}, \tag{12}$$

Constrained Policy Optimization paper review - again

CPO = trust region method + constraint

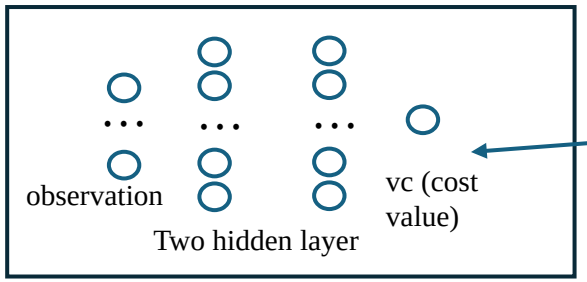
Constrained Policy Optimization paper review - again





ret

train



cret

train

train

pi_loss
v_loss
vc_loss

Calculated at last local steps of epoch
by interacting with env

Constrained Policy Optimization paper review - again

Large or continuous MDP → solving exact optimal policy is intractable

CMDP

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J(\pi)$$
$$\text{s.t. } J_{C_i}(\pi) \leq d_i \quad i = 1, ..., m$$
$$D(\pi, \pi_k) \leq \delta.$$

CPO

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} [A^{\pi_k}(s,a)]$$
$$\text{s.t. } J_{C_i}(\pi_k) + \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} \left[A^{\pi_k}_{C_i}(s,a)\right] \leq d_i \quad \forall i$$
$$\bar{D}_{KL}(\pi || \pi_k) \leq \delta.$$

(10)

$$\theta_{k+1} = \arg\max_{\theta} \; g^T(\theta - \theta_k)$$
$$\text{s.t. } c_i + b_i^T(\theta - \theta_k) \leq 0 \quad i = 1, ..., m$$
$$\frac{1}{2}(\theta - \theta_k)^T H (\theta - \theta_k) \leq \delta.$$

(11)

$$\max_{\substack{\lambda \geq 0 \\ \nu \succeq 0}} \frac{-1}{2\lambda}\left(g^T H^{-1} g - 2r^T \nu + \nu^T S \nu\right) + \nu^T c - \frac{\lambda\delta}{2},$$

(12)

Algorithm 1 Constrained Policy Optimization

Input: Initial policy $\pi_0 \in \Pi_\theta$ tolerance α
for $k = 0, 1, 2, ...$ **do**
  Sample a set of trajectories $\mathcal{D} = \{\tau\} \sim \pi_k = \pi(\theta_k)$
  Form sample estimates $\hat{g}, \hat{b}, \hat{H}, \hat{c}$ with $\mathcal{D}$
  if approximate CPO is feasible **then**
    Solve dual problem (12) for λ_k^*, ν_k^*
    Compute policy proposal θ^* with (13)
  else
    Compute recovery policy proposal θ^* with (14)
  end if
  Obtain θ_{k+1} by backtracking linesearch to enforce satisfaction of sample estimates of constraints in (10)
end for

Constrained Policy Optimization paper review – again

- MDP

Goal: improve performance(goal) + satisfy constraint

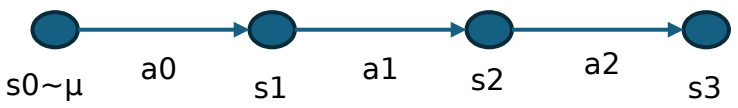
$R : S \times A \times S \to \mathbb{R}$ is the reward function

$P : S \times A \times S \to [0, 1]$ is the transition probability function

$P(s'|s, a)$: probability of transition to state s' given that
previous state s
agent took action a in s

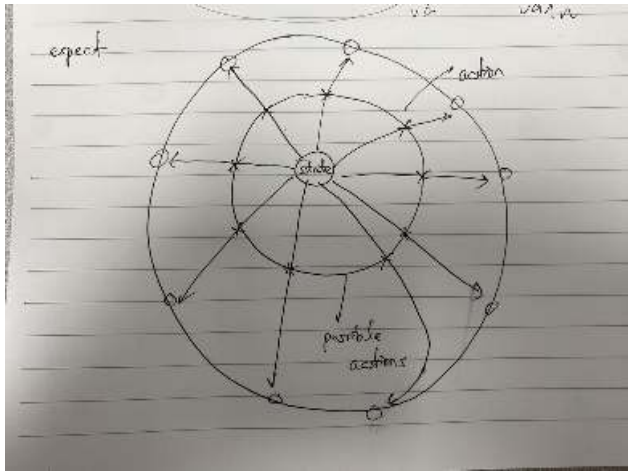
$\pi : S \to \mathcal{P}(A)$: stationary policy,
Map from states to probability distributions over action

$$J(\pi) \doteq \mathop{\mathrm{E}}_{\tau \sim \pi} [\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})].$$

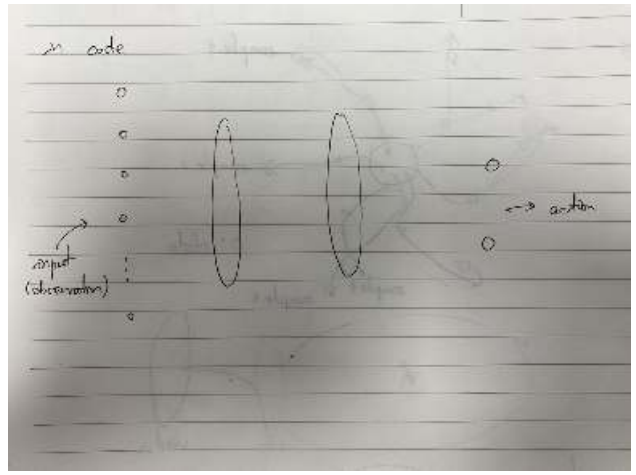


$a_t \sim \pi(\cdot\,|s_t)$

$a_{t+1} \sim P(\cdot\,|s_t, a_t)$



Expect: policy is composed with enormous states



In code: policy is neural network itself

$V^{\pi}(s) \doteq \mathrm{E}_{\tau \sim \pi}[R(\tau)|s_0 = s]$: On-policy value function

$Q^{\pi}(s, a) \doteq \mathrm{E}_{\tau \sim \pi}[\mathrm{R}(\tau)|s_0 = s, a_0 = a]$: on-policy action-value function

$A^{\pi}(s, a) \doteq Q^{\pi}(s, a) - V^{\pi}(s)$: advantage function

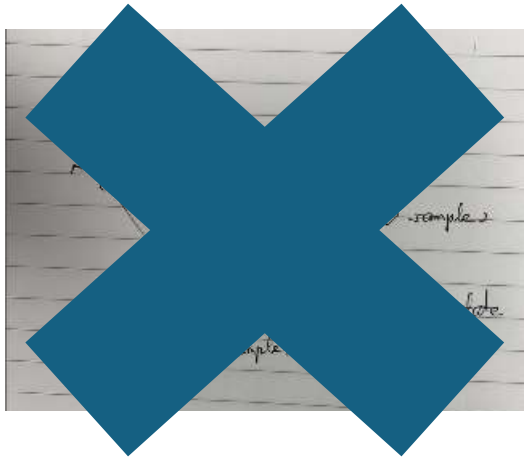
Detailed instruction to evaluate V, Q, A might be in TRPO paper

Constrained Policy Optimization paper review – again

- CMDP

Discounted future state distribution: $d^{\pi}(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^{t}P(s_t = s|\pi).$

Difference in performance btw two policies $J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathop{\mathrm{E}}_{\substack{s\sim d^{\pi'}\\ a\sim\pi'}}\left[A^{\pi}(s,a)\right],$



Performance difference is calculated based on KL-divergence

Constrained Markov Decision Processes

Augmented cost functions $C_i : S \times A \times S \to \mathbb{R}$

Expected discounted return of policy π w.r.t. cost function Ci: C_i: $J_{C_i}(\pi) = \mathop{\mathrm{E}}_{\tau\sim\pi}[\sum_{t=0}^{\infty}\gamma^{t}C_i(s_t, a_t, s_{t+1})].$

Set of feasible stationary policies for CMDP: $\Pi_C \doteq \{\pi \in \Pi \ : \ \forall i, J_{C_i}(\pi) \leq d_i\}$

RL problem in CMDP: $\pi^* = \arg\max_{\pi\in\Pi_C} J(\pi).$

i	1	2	3	4
v_i	10	40	30	50
w_i	5	4	6	3

$V[i,w]$	0	1	2	3	4	5	6	7	8	9	10
$i=0$	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	10	10	10	10	10	10
2	0	0	0	0	40	40	40	40	40	50	50
3	0	0	0	0	40	40	40	40	40	50	70
4	0	0	0	50	50	50	50	90	90	90	90

Constrained Policy Optimization paper review – again

- Policy update

neural networks of a fixed architecture).

Large or continuous MDP → solving exact optimal policy is intractable

search optimal policy within $\Pi_\theta \subseteq \Pi$ (parametrized policies w/ parameters θ)

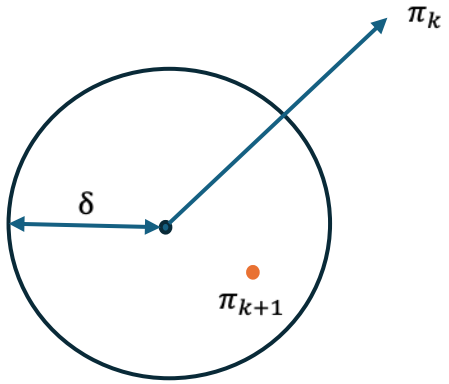
iteratively update policy by maximizing J(π) over a local neighborhood of the most recent iterate π_k

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J(\pi)$$
$$\text{s.t. } D(\pi, \pi_k) \leq \delta,$$

Local policy search update of MDP

Objective is estimated by linearizing around π_k as $J(\pi_k) + g^T(\theta - \theta_k)$, g is the policy gradient.

Standard policy gradient update: $D(\pi, \pi_k) = \|\theta - \theta_k\|_2$



Such like distance, but not a distance measure exactly

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J(\pi)$$
$$\text{s.t. } J_{C_i}(\pi) \leq d_i \quad i = 1, ..., m$$
$$D(\pi, \pi_k) \leq \delta.$$

Local policy search update of CMDP

Hard to implement → develop principled approximation with particular choice of D,

Replace objective and constraints w/ surrogate function

Can bound update's worst case performance and worst-case constraint violation

Constrained Policy Optimization paper review – again

- What is constraint?

In experiment, only one constraint: safe states / unsafe states

- agent suffers a safety cost of 1 for being in an unsafe state

various approximations between (3) and practical algorithm → need factor of safety to minimize the chance of constraint violations.

$$C_i^+(s,a,s') = C_i(s,a,s') + \Delta_i(s,a,s'), \qquad (15)$$

Δ: probability of entering an unsafe state within a fixed time horizon according to a learned model (updated at each iteration)

```
def update():
    cur_cost = logger.get_stats('EpCost')[0]
    c = cur_cost - cost_lim
    if c > 0 and agent.cares_about_cost:
        logger.log('Warning! Safety constraint is already violated.', 'red')
```

```
# Cost constraints / penalties:
cost_lim=25,
penalty_init=1.,
penalty_lr=5e-2,
```

```
for epoch in range(epochs):

    if agent.use_penalty:
        cur_penalty = sess.run(penalty)

    for t in range(local_steps_per_epoch):

        # Possibly render
        if render and proc_id()==0: # and t < 1000:
            env.render()

        # Get outputs from policy
        get_action_outs = sess.run(get_action_ops,
                                   feed_dict={x_ph: o[np.newaxis]})
        a = get_action_outs['pi']
        v_t = get_action_outs['v']
        vc_t = get_action_outs.get('vc', 0)  # Agent may not use cost value func
        logp_t = get_action_outs['logp_pi']
        pi_info_t = get_action_outs['pi_info']

        # Step in environment
        o2, r, d, info = env.step(a)

        # Include penalty on cost
        c = info.get('cost', 0)
```

```
886.0
887.0
888.0
889.0
890.0
891.0
892.0
893.0
894.0
895.0
896.0
897.0
898.0
899.0
900.0
901.0
902.0
903.0
904.0
904.0
904.0
904.0
904.0
904.0
904.0
904.0
904.0
904.0
```

Constrained Policy Optimization paper review – again

- Policy performance bound

What prove

1. new bounds on the difference in returns btw two arbitrary stochastic policies in terms of an avg divergence btw them
2. Prove worst-case performance degradation at each update

Theorem 1. *For any function $f : S \to \mathbb{R}$ and any policies π' and π, define $\delta_f(s, a, s') \doteq R(s, a, s') + \gamma f(s') - f(s)$,*

$$\epsilon_f^{\pi'} \doteq \max_s \left| \mathrm{E}_{a \sim \pi', s' \sim P}[\delta_f(s, a, s')] \right| ,$$

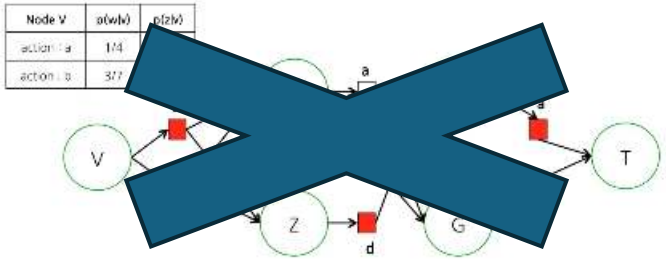
$$L_{\pi,f}(\pi') \doteq \mathop{\mathrm{E}}_{\substack{s \sim d^\pi \\ a \sim \pi \\ s' \sim P}} \left[\left(\frac{\pi'(a|s)}{\pi(a|s)} - 1 \right) \delta_f(s, a, s') \right] , \text{ and}$$

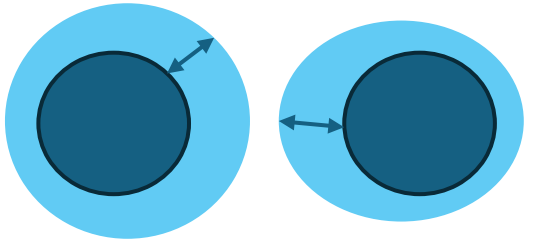
$$D^{\pm}_{\pi,f}(\pi') \doteq \frac{L_{\pi,f}(\pi')}{1-\gamma} \pm \frac{2\gamma\epsilon_f^{\pi'}}{(1-\gamma)^2} \mathop{\mathrm{E}}_{s \sim d^\pi} \left[D_{TV}(\pi' \| \pi)[s] \right] ,$$

where $D_{TV}(\pi' \| \pi)[s] = (1/2) \sum_a |\pi'(a|s) - \pi(a|s)|$ is the total variational divergence between action distributions at s. The following bounds hold:

$$D^{+}_{\pi,f}(\pi') \geq J(\pi') - J(\pi) \geq D^{-}_{\pi,f}(\pi'). \qquad (4)$$

Furthermore, the bounds are tight (when $\pi' = \pi$, all three expressions are identically zero).





Constrained Policy Optimization paper review – again

- Policy performance bound

Corollary 1. *For any policies π', π, with $\epsilon^{\pi'} \doteq \max_s |\mathrm{E}_{a\sim\pi'}[A^\pi(s,a)]|$, the following bound holds:*

$$J(\pi') - J(\pi) \geq \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s\sim d^\pi \\ a\sim\pi'}} \left[A^\pi(s,a) - \frac{2\gamma\epsilon^{\pi'}}{1-\gamma} D_{TV}(\pi'||\pi)[s] \right]. \quad (5)$$

The bound (5) should be compared with equation (1). The term $(1-\gamma)^{-1}\mathrm{E}_{s\sim d^\pi, a\sim\pi'}[A^\pi(s,a)]$ in (5) is an approximation to $J(\pi') - J(\pi)$, using the state distribution d^π instead of $d^{\pi'}$, which is known to equal $J(\pi') - J(\pi)$ to first order in the parameters of π' on a neighborhood around π (Kakade & Langford, 2002). The bound can therefore be viewed as describing the worst-case approximation error, and it justifies using the approximation as a *surrogate* for $J(\pi') - J(\pi)$.

Corollary 2. *For any policies π', π, and any cost function C_i, with $\epsilon^{\pi'}_{C_i} \doteq \max_s |\mathrm{E}_{a\sim\pi'}[A^\pi_{C_i}(s,a)]|$, the following bound holds:*

$$J_{C_i}(\pi') - J_{C_i}(\pi) \leq \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s\sim d^\pi \\ a\sim\pi'}} \left[A^\pi_{C_i}(s,a) + \frac{2\gamma\epsilon^{\pi'}_{C_i}}{1-\gamma} D_{TV}(\pi'||\pi)[s] \right]. \quad (6)$$

The bounds we have given so far are in terms of the TV-divergence between policies, but trust region methods constrain the KL-divergence between policies, so bounds that connect performance to the KL-divergence are desirable. We make the connection through Pinsker's in-

Corollary 3. *In bounds (4), (5), and (6), make the substitution*

$$\mathop{\mathrm{E}}_{s\sim d^\pi}[D_{TV}(\pi'||\pi)[s]] \to \sqrt{\frac{1}{2} \mathop{\mathrm{E}}_{s\sim d^\pi}[D_{KL}(\pi'||\pi)[s]]}.$$

The resulting bounds hold.

Cause we will use Trust Region method

Constrained Policy Optimization paper review – again

- Trust region method

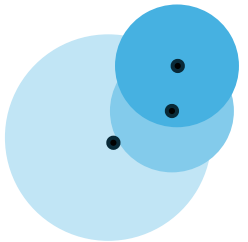
Motivation: approximate lower bound on policy performance given in

$$J(\pi') - J(\pi) \geq \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s\sim d^{\pi} \\ a\sim\pi'}} \left[A^{\pi}(s,a) - \frac{2\gamma\epsilon^{\pi'}}{1-\gamma} D_{TV}(\pi' || \pi)[s] \right]. \quad (5)$$

Trust Region Methods:

$$\pi_{k+1} = \arg\max_{\pi\in\Pi_\theta} \mathop{\mathrm{E}}_{\substack{s\sim d^{\pi_k} \\ a\sim\pi}} [A^{\pi_k}(s,a)] \quad \text{s.t. } \bar{D}_{KL}(\pi||\pi_k) \leq \delta, \quad (8)$$

where $\bar{D}_{KL}(\pi||\pi_k) = \mathrm{E}_{s\sim\pi_k}[D_{KL}(\pi||\pi_k)[s]]$, and $\delta > 0$ is the step size. The set $\{\pi_\theta \in \Pi_\theta : \bar{D}_{KL}(\pi||\pi_k) \leq \delta\}$ is called the *trust region*.



KL-divergence is not a distance measure exactly

This method guarantee monotonic performance improvements

Lower bound on the policy performance difference between π_k *and* π_{k+1} *is*

$$J(\pi_{k+1}) - J(\pi_k) \geq \frac{-\sqrt{2\delta}\gamma\epsilon^{\pi_{k+1}}}{(1-\gamma)^2}, \quad (9)$$

where $\epsilon^{\pi_{k+1}} = \max_s \left| \mathrm{E}_{a\sim\pi_{k+1}} [A^{\pi_k}(s,a)] \right|$.

Constrained Policy Optimization paper review – again

- How to solve?

CMDP

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J(\pi)$$
$$\text{s.t. } J_{C_i}(\pi) \leq d_i \quad i = 1, ..., m$$
$$D(\pi, \pi_k) \leq \delta.$$

Difficult to implement in practice
∵ requires evaluation of the constraint functions to determine π is feasible.

Policy search update for CMDPs
alleviates off-policy evaluation problem

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} [A^{\pi_k}(s,a)] - \alpha_k \sqrt{\bar{D}_{KL}(\pi || \pi_k)}$$
$$\text{s.t. } J_{C_i}(\pi_k) + \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} \left[\frac{A^{\pi_k}_{C_i}(s,a)}{1-\gamma} \right] + \beta^i_k \sqrt{\bar{D}_{KL}(\pi || \pi_k)} \leq d_i$$

Theoretically guaranteed update take too small step

CPO

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} [A^{\pi_k}(s,a)]$$
$$\text{s.t. } J_{C_i}(\pi_k) + \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} [A^{\pi_k}_{C_i}(s,a)] \leq d_i \quad \forall i$$
$$\bar{D}_{KL}(\pi || \pi_k) \leq \delta.$$

use trust region instead of penalties on policy divergence to enable larger step size

Constrained Policy Optimization paper review – again

- How to solve?

CPO

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} \mathop{\mathrm{E}}_{\substack{s\sim d^{\pi_k} \\ a\sim\pi}} [A^{\pi_k}(s,a)]$$

$$\text{s.t. } J_{C_i}(\pi_k) + \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s\sim d^{\pi_k} \\ a\sim\pi}} \left[A^{\pi_k}_{C_i}(s,a)\right] \le d_i \quad \forall i$$

$$\bar{D}_{KL}(\pi || \pi_k) \le \delta. \tag{10}$$

Policies with high-dimensional parameter spaces like neural networks
→ (10) is impractical to solve directly because of the computational cost

↓

For small step sizes δ
objective and cost constraints are well-approximated by linearizing around π_k

$$\theta_{k+1} = \arg\max_\theta \; g^T(\theta - \theta_k)$$

$$\text{s.t.} \quad c_i + b_i^T(\theta - \theta_k) \le 0 \quad i = 1, ..., m$$

$$\frac{1}{2}(\theta - \theta_k)^T H (\theta - \theta_k) \le \delta. \tag{11}$$

↓

Dual problem

$$\max_{\substack{\lambda \ge 0 \\ \nu \succeq 0}} \frac{-1}{2\lambda}\left(g^T H^{-1} g - 2r^T\nu + \nu^T S \nu\right) + \nu^T c - \frac{\lambda\delta}{2}, \tag{12}$$

Approximation error →
add recovery / backtracking linear search step →

Algorithm 1 Constrained Policy Optimization

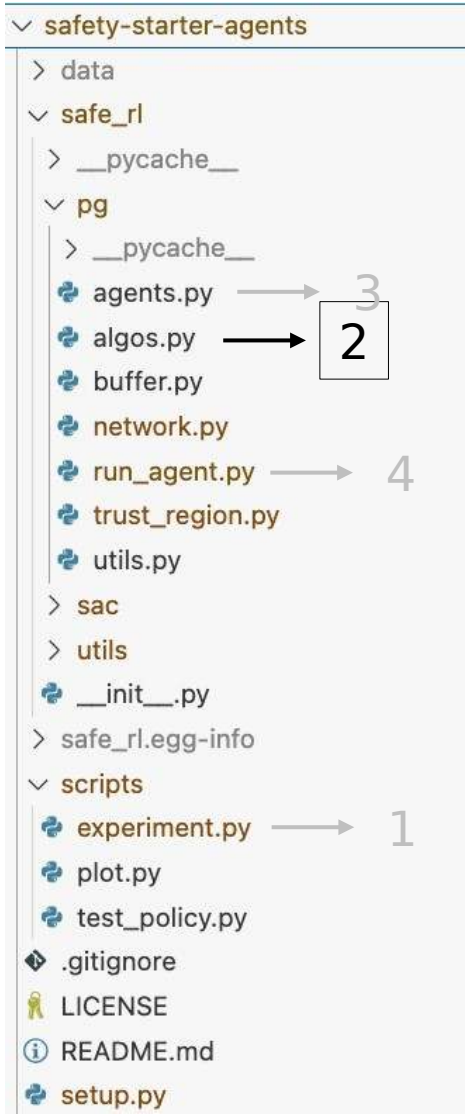
Input: Initial policy $\pi_0 \in \Pi_\theta$ tolerance α
for $k = 0, 1, 2, ...$ **do**
  Sample a set of trajectories $\mathcal{D} = \{\tau\} \sim \pi_k = \pi(\theta_k)$
  Form sample estimates $\hat{g}, \hat{b}, \hat{H}, \hat{c}$ with $\mathcal{D}$
  if approximate CPO is feasible **then**
    Solve dual problem (12) for λ_k^*, ν_k^*
    Compute policy proposal θ^* with (13)
  else
    Compute recovery policy proposal θ^* with (14)
  end if
  Obtain θ_{k+1} by backtracking linesearch to enforce satisfaction of sample estimates of constraints in (10)
end for

Code review

Code review - Hierarchy



Code review - Algos.py



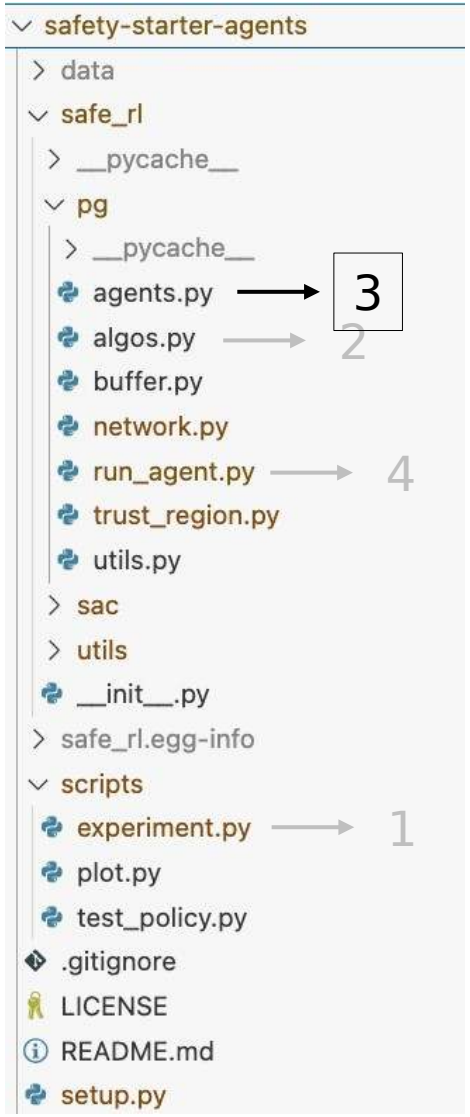

```
algo(env_fn=lambda: gym.make(env_name),
     ac_kwargs=dict(
         hidden_sizes=(256, 256),
        ),
     epochs=epochs,
     steps_per_epoch=steps_per_epoch,
     save_freq=save_freq,
     target_kl=target_kl,
     cost_lim=cost_lim,
     seed=seed,
     logger_kwargs=logger_kwargs
     )
```

직전 코드

```
from functools import partial
from safe_rl.pg.agents import PPOAgent, TRPOAgent, CPOAgent
from safe_rl.pg.run_agent import run_polopt_agent
def cpo(**kwargs):
    cpo_kwargs = dict(
                      reward_penalized=False,  # Irrelevant in CPO
                      objective_penalized=False,  # Irrelevant in CPO
                      learn_penalty=False,  # Irrelevant in CPO
                      penalty_param_loss=False  # Irrelevant in CPO
                      )
    agent = CPOAgent(**cpo_kwargs)
    run_polopt_agent(agent=agent, **kwargs)
```

CPO Agent 객체 생성

Code review - Agents.py

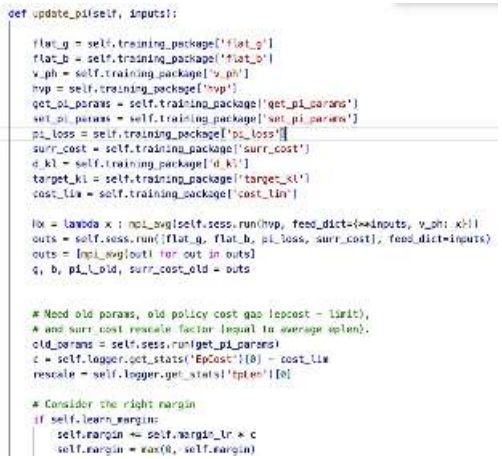



```python
from functools import partial
from safe_rl.pg.agents import PPOAgent, TRPOAgent, CPOAgent
from safe_rl.pg.run_agent import run_polopt_agent
def cpo(**kwargs):
    cpo_kwargs = dict(
                      reward_penalized=False,  # Irrelevant in CPO
                      objective_penalized=False,  # Irrelevant in CPO
                      learn_penalty=False,  # Irrelevant in CPO
                      penalty_param_loss=False  # Irrelevant in CPO
                      )
    agent = CPOAgent(**cpo_kwargs)
    run_polopt_agent(agent=agent, **kwargs)
```

CPO Agent 객체 생성

```python
class CPOAgent(TrustRegionAgent):

    def __init__(self, learn_margin=False, **kwargs):
        super().__init__(**kwargs)
        self.learn_margin = learn_margin
        self.params.update(dict(
            constrained=True,
            save_penalty=True
            ))
        self.margin = 0
        self.margin_lr = 0.05
```



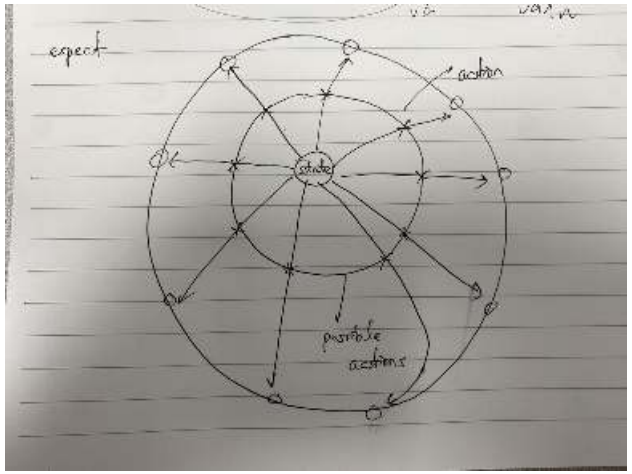
See later

Code review – How to sample?

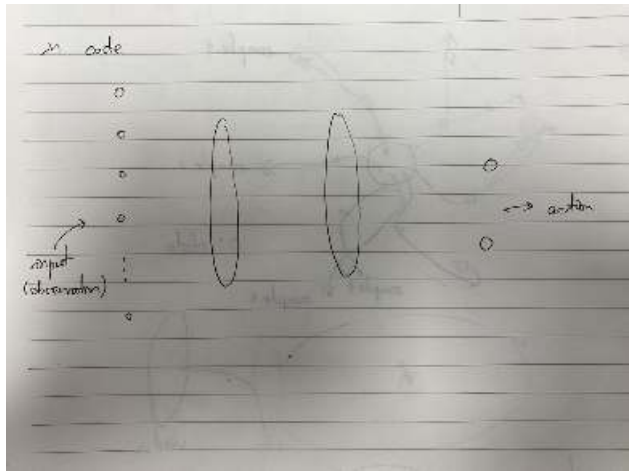
Getting policy difference ≠ sampling

Reducing policy difference

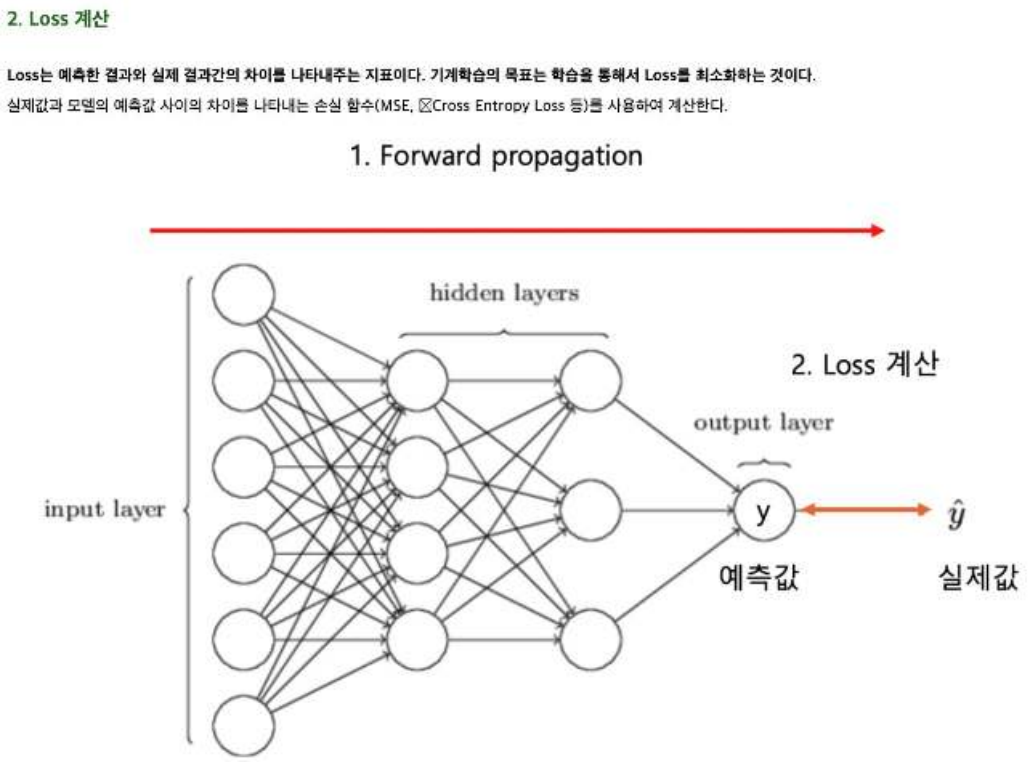
$$J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s\sim d^{\pi'} \\ a\sim\pi'}} [A^{\pi}(s,a)],$$



Expect: policy is composed with enormous states



In code: policy is neural network itself



Code review – How to sample?

Sampling by concurrently running multiple

```
#=====================================================================#
#  Cumulative cost calculations                                       #
#=====================================================================#
cumulative_cost = mpi_sum(cum_cost)
cost_rate = cumulative_cost / ((epoch+1)*steps_per_epoch)
```

test.py > ...

```python
from mpi4py import MPI
import numpy as np

def mpi_avg(x):
    comm = MPI.COMM_WORLD

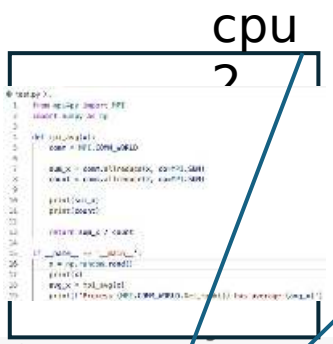
    sum_x = comm.allreduce(x, op=MPI.SUM)
    count = comm.allreduce(2, op=MPI.SUM)

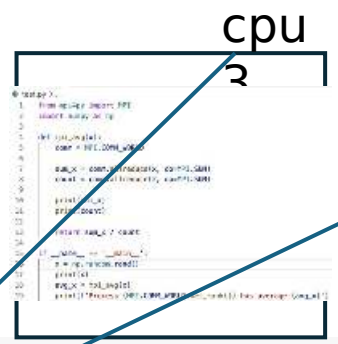
    print(sum_x)
    print(count)

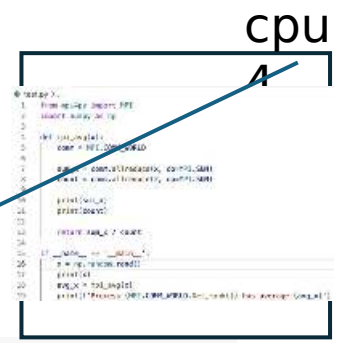
    return sum_x / count

if __name__ == '__main__':
    x = np.random.rand()
    print(x)
    avg_x = mpi_avg(x)
    print(f'Process {MPI.COMM_WORLD.Get_rank()} has average {avg_x}')
```










```
(test) hanes1207@EuiseokHan-MacbookAir 코딩연습 % mpirun -np 4 python test.py
0.1228208748390561 3
0.83762632866493
0.23815524526583698
0.2912635921277599
1.489866040897583
8
1.489866040897583
8
Process 2 has average 0.18623325511219788
1.489866040897583
8
Process 1 has average 0.18623325511219788
1.489866040897583
8
Process 3 has average 0.18623325511219788
Process 0 has average 0.18623325511219788
```

Code review – How to sample?

Local steps are batch!

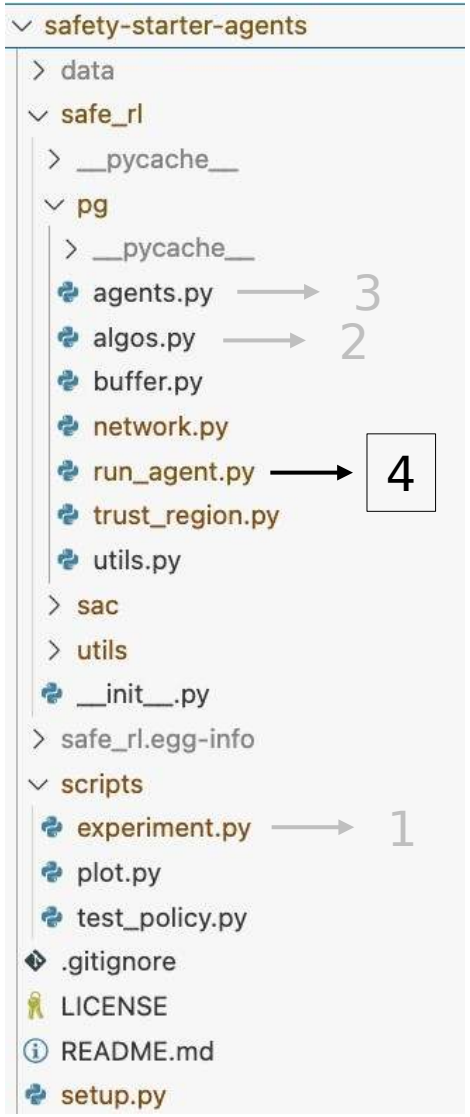
```
if terminal or (t==local_steps_per_epoch-1):

    # If trajectory didn't reach terminal state, bootstrap value target(s)
    if d and not(ep_len == max_ep_len):
        # Note: we do not count env time out as true terminal state
        last_val, last_cval = 0, 0
    else:
        feed_dict={x_ph: o[np.newaxis]}
        if agent.reward_penalized:
            last_val = sess.run(v, feed_dict=feed_dict)
            last_cval = 0
        else:
            last_val, last_cval = sess.run([v, vc], feed_dict=feed_dict)
    buf.finish_path(last_val, last_cval)

    # Only save EpRet / EpLen if trajectory finished
    if terminal:
        logger.store(EpRet=ep_ret, EpLen=ep_len, EpCost=ep_cost)
    else:
        print('Warning: trajectory cut off by epoch at %d steps.'%ep_len)

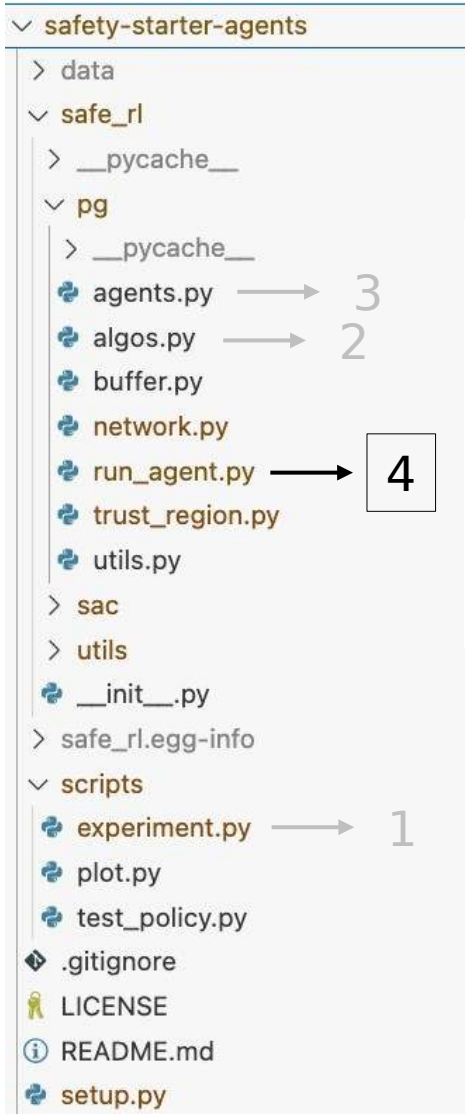
    # Reset environment
    o, r, d, c, ep_ret, ep_len, ep_cost = env.reset(), 0, False, 0, 0, 0, 0
```

Code review – Run_agent.py




```python
# Multi-purpose agent runner for policy optimization algos
# (PPO, TRPO, their primal-dual equivalents, CPO)
def run_polopt_agent(env_fn,
                     agent=PPOAgent(),
                     actor_critic=mlp_actor_critic,
                     ac_kwargs=dict(),
                     seed=0,
                     render=False,
                     # Experience collection:
                     steps_per_epoch=4000,
                     epochs=50,
                     max_ep_len=1000,
                     # Discount factors:
                     gamma=0.99,
                     lam=0.97,
                     cost_gamma=0.99,
                     cost_lam=0.97,
                     # Policy learning:
                     ent_reg=0.,
                     # Cost constraints / penalties:
                     cost_lim=25,
                     penalty_init=1.,
                     penalty_lr=5e-2,
                     # KL divergence:
                     target_kl=0.01,
                     # Value learning:
                     vf_lr=1e-3,
                     vf_iters=80,
                     # Logging:
                     logger=None,
                     logger_kwargs=dict(),
                     save_freq=1
                     ):
```

Code review – Run_agent.py




```
#=====================================================================#
#  Prepare logger, seed, and environment in this process              #
#=====================================================================#

logger = EpochLogger(**logger_kwargs) if logger is None else logger
logger.save_config(locals())

seed += 10000 * proc_id()
tf.set_random_seed(seed)
np.random.seed(seed)

env = env_fn()

agent.set_logger(logger)
```

```
#=====================================================================#
#  Create computation graph for actor and critic (not training routine)   #
#=====================================================================#

# Share information about action space with policy architecture
ac_kwargs['action_space'] = env.action_space

# Inputs to computation graph from environment spaces
x_ph, a_ph = placeholders_from_spaces(env.observation_space, env.action_space)

# Inputs to computation graph for batch data
adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph = placeholders(*(None for _ in range(5)))

# Inputs to computation graph for special purposes
surr_cost_rescale_ph = tf.placeholder(tf.float32, shape=())
cur_cost_ph = tf.placeholder(tf.float32, shape=())

# Outputs from actor critic
ac_outs = actor_critic(x_ph, a_ph, **ac_kwargs)
pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent, v, vc = ac_outs

# Organize placeholders for zipping with data from buffer on updates
buf_phs = [x_ph, a_ph, adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph]
buf_phs += values_as_sorted_list(pi_info_phs)

# Organize symbols we have to compute at each step of acting in env
get_action_ops = dict(pi=pi,
                      v=v,
                      logp_pi=logp_pi,
                      pi_info=pi_info)

# If agent is reward penalized, it doesn't use a separate value function
# for costs and we don't need to include it in get_action_ops; otherwise we do.
if not(agent.reward_penalized):
    get_action_ops['vc'] = vc

# Count variables
var_counts = tuple(count_vars(scope) for scope in ['pi', 'vf', 'vc'])
logger.log('\nNumber of parameters: \t pi: %d, \t v: %d, \t vc: %d\n'%var_counts)

# Make a sample estimate for entropy to use as sanity check
approx_ent = tf.reduce_mean(-logp)
```

Code review – Run_agent.py

```
#=====================================================================#
#  Create computation graph for actor and critic (not training routine)   #
#=====================================================================#

# Share information about action space with policy architecture
ac_kwargs['action_space'] = env.action_space

# Inputs to computation graph from environment spaces
x_ph, a_ph = placeholders_from_spaces(env.observation_space, env.action_space)

# Inputs to computation graph for batch data
adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph = placeholders(*(None for _ in range(5)))

# Inputs to computation graph for special purposes
surr_cost_rescale_ph = tf.placeholder(tf.float32, shape=())
cur_cost_ph = tf.placeholder(tf.float32, shape=())

# Outputs from actor critic
ac_outs = actor_critic(x_ph, a_ph, **ac_kwargs)
pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent, v, vc = ac_outs

# Organize placeholders for zipping with data from buffer on updates
buf_phs = [x_ph, a_ph, adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph]
buf_phs += values_as_sorted_list(pi_info_phs)

# Organize symbols we have to compute at each step of acting in env
get_action_ops = dict(pi=pi,
                      v=v,
                      logp_pi=logp_pi,
                      pi_info=pi_info)

# If agent is reward penalized, it doesn't use a separate value function
# for costs and we don't need to include it in get_action_ops; otherwise we do.
if not(agent.reward_penalized):
    get_action_ops['vc'] = vc

# Count variables
var_counts = tuple(count_vars(scope) for scope in ['pi', 'vf', 'vc'])
logger.log('\nNumber of parameters: \t pi: %d, \t v: %d, \t vc: %d\n'%var_counts)

# Make a sample estimate for entropy to use as sanity check
approx_ent = tf.reduce_mean(-logp)
```

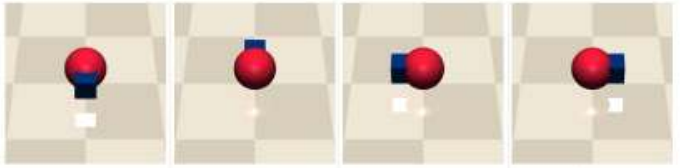
```
class Engine(gym.Env, gym.utils.EzPickle):

    def __init__(self, config={}):
        # First, parse configuration. Important note: LOTS of stuff happens in
        # parse, and many attributes of the class get set through setattr. If you
        # are trying to track down where an attribute gets initially set, and
        # can't find it anywhere else, it's probably set via the config dict
        # and this parse function.
        self.parse(config)
        gym.utils.EzPickle.__init__(self, config=config)

        # Load up a simulation of the robot, just to figure out observation space
        self.robot = Robot(self.robot_base)

        self.action_space = gym.spaces.Box(-1, 1, (self.robot.nu,), dtype=np.float32)
```

Point



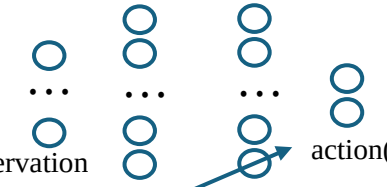
A simple robot constrained to a 2D plane has two actuators, one for rotation and the other for forward/backward movement. This decomposed control scheme makes it particularly easy to control the robot's navigation. It has a small square in front of it, which makes it easier to visually determine the robot's orientation and also helps Point push the box that appears in Push.

Specific Action Space	Box(-1.0, 1.0, (2,), float64)
Specific Observation Shape	(12,)
Specific Observation High	inf
Specific Observation Low	-inf

Specific Action Space

Num	Action	Control Min	Control Max	Name (in corresponding XML file)	Joint/Site	Unit
0	force applied on the agent to move forward or backward	0	1	x	site	force (N)
1	velocity of the agent, which is around the z-axis.	0	1	z	hinge	velocity (m/s)

Run_agent.py




```
#=========================================================================#
#  Create computation graph for actor and critic (not training routine)   #
#=========================================================================#

# Share information about action space with policy architecture
ac_kwargs['action_space'] = env.action_space

# Inputs to computation graph from environment spaces
x_ph, a_ph = placeholders_from_spaces(env.observation_space, env.action_space)

# Inputs to computation graph for batch data
adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph = placeholders(*(None for _ in range(5)))

# Inputs to computation graph for special purposes
surr_cost_rescale_ph = tf.placeholder(tf.float32, shape=())
cur_cost_ph = tf.placeholder(tf.float32, shape=())

# Outputs from actor critic
ac_outs = actor_critic(x_ph, a_ph, **ac_kwargs)
pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent, v, vc = ac_outs
```

```
"""
Actor-Critics
"""
def mlp_actor_critic(x, a, hidden_sizes=(64,64), activation=tf.tanh,
                     output_activation=None, policy=None, action_space=None):

    # default policy builder depends on action space
    if policy is None and isinstance(action_space, Box):
        policy = mlp_gaussian_policy
    elif policy is None and isinstance(action_space, Discrete):
        policy = mlp_categorical_policy

    with tf.variable_scope('pi'):
        policy_outs = policy(x, a, hidden_sizes, activation, output_activation, action_space)
        pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent = policy_outs

    with tf.variable_scope('vf'):
        v = tf.squeeze(mlp(x, list(hidden_sizes)+[1], activation, None), axis=1)

    with tf.variable_scope('vc'):
        vc = tf.squeeze(mlp(x, list(hidden_sizes)+[1], activation, None), axis=1)

    return pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent, v, vc
```

```
def mlp_gaussian_policy(x, a, hidden_sizes, activation, output_activation, action_space):
    act_dim = a.shape.as_list()[-1]
    mu = mlp(x, list(hidden_sizes)+[act_dim], activation, output_activation)
    log_std = tf.get_variable(name='log_std', initializer=-0.5*np.ones(act_dim, dtype=np.float32))
    std = tf.exp(log_std)
    pi = mu + tf.random_normal(tf.shape(mu)) * std
    logp = gaussian_likelihood(a, mu, log_std)
    logp_pi = gaussian_likelihood(pi, mu, log_std)

    old_mu_ph, old_log_std_ph = placeholders(act_dim, act_dim)
    d_kl = gaussian_kl(mu, log_std, old_mu_ph, old_log_std_ph)
    ent = gaussian_entropy(log_std)

    pi_info = {'mu': mu, 'log_std': log_std}
    pi_info_phs = {'mu': old_mu_ph, 'log_std': old_log_std_ph}

    return pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent
```

```
def mlp_categorical_policy(x, a, hidden_sizes, activation, output_activation, action_space):
    act_dim = action_space.n
    logits = mlp(x, list(hidden_sizes)+[act_dim], activation, None)
    logp_all = tf.nn.log_softmax(logits)
    pi = tf.squeeze(tf.multinomial(logits,1), axis=1)
    logp = tf.reduce_sum(tf.one_hot(a, depth=act_dim) * logp_all, axis=1)
    logp_pi = tf.reduce_sum(tf.one_hot(pi, depth=act_dim) * logp_all, axis=1)

    old_logp_all = placeholder(act_dim)
    d_kl = categorical_kl(logp_all, old_logp_all)
    ent = categorical_entropy(logp_all)

    pi_info = {'logp_all': logp_all}
    pi_info_phs = {'logp_all': old_logp_all}

    return pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent
```

Run_agent.py

```
#=========================================================================#
#  Create computation graph for actor and critic (not training routine)   #
#=========================================================================#

# Share information about action space with policy architecture
ac_kwargs['action_space'] = env.action_space

# Inputs to computation graph from environment spaces
x_ph, a_ph = placeholders_from_spaces(env.observation_space, env.action_space)

# Inputs to computation graph for batch data
adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph = placeholders(*(None for _ in range(5)))

# Inputs to computation graph for special purposes
surr_cost_rescale_ph = tf.placeholder(tf.float32, shape=())
cur_cost_ph = tf.placeholder(tf.float32, shape=())

# Outputs from actor critic
ac_outs = actor_critic(x_ph, a_ph, **ac_kwargs)
pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent, v, vc = ac_outs

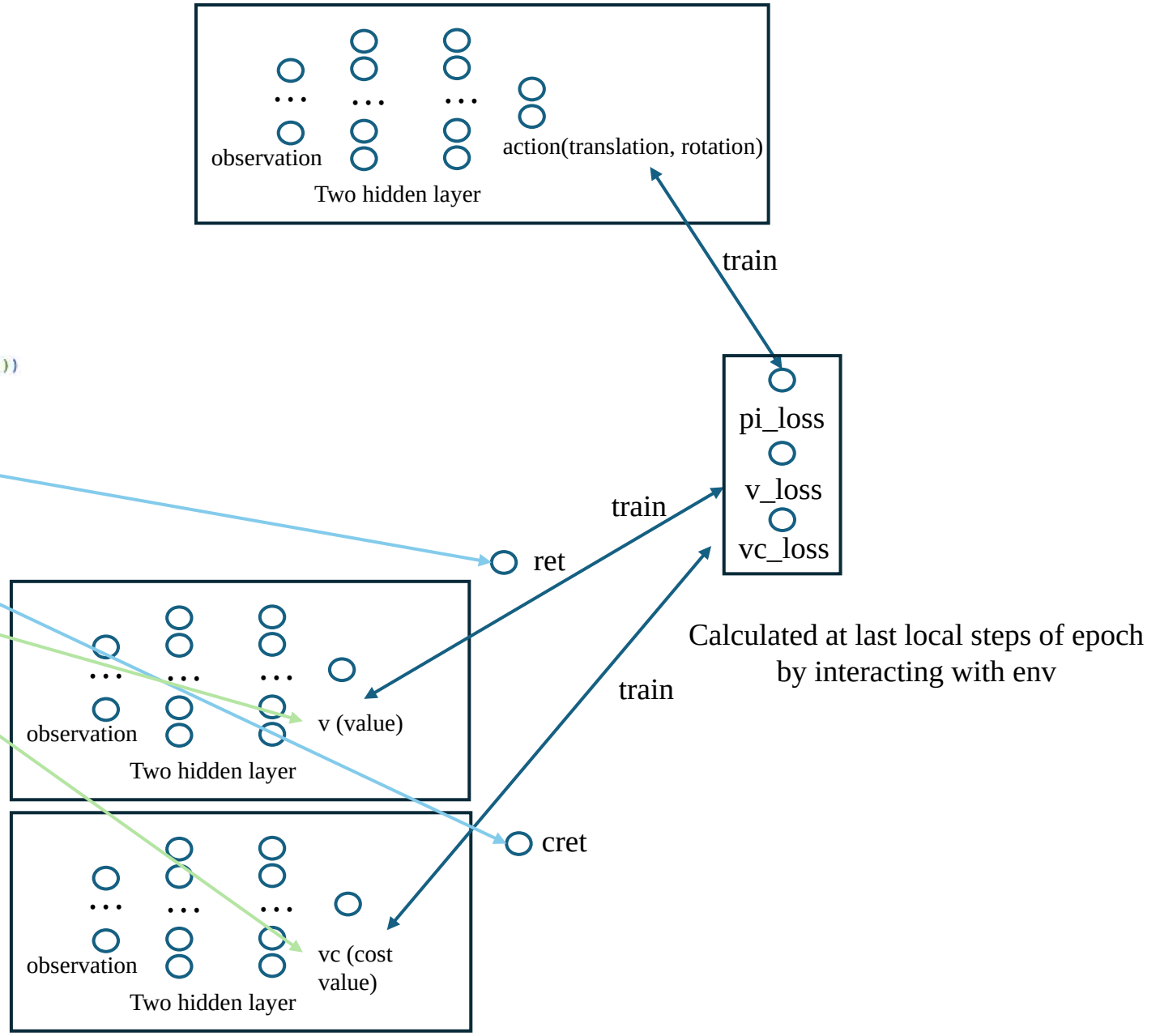
# Organize placeholders for zipping with data from buffer on updates
buf_phs = [x_ph, a_ph, adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph]
buf_phs += values_as_sorted_list(pi_info_phs)

# Organize symbols we have to compute at each step of acting in env
get_action_ops = dict(pi=pi,
                      v=v,
                      logp_pi=logp_pi,
                      pi_info=pi_info)

# If agent is reward penalized, it doesn't use a separate value function
# for costs and we don't need to include it in get_action_ops; otherwise we do.
if not(agent.reward_penalized):
    get_action_ops['vc'] = vc

# Count variables
var_counts = tuple(count_vars(scope) for scope in ['pi', 'vf', 'vc'])
logger.log('\nNumber of parameters: \t pi: %d, \t v: %d, \t vc: %d\n'%var_counts)

# Make a sample estimate for entropy to use as sanity check
approx_ent = tf.reduce_mean(-logp)
```



Run_agent.py

```python
"""
Actor-Critics
"""
def mlp_actor_critic(x, a, hidden_sizes=(64,64), activation=tf.tanh,
                     output_activation=None, policy=None, action_space=None):

    # default policy builder depends on action space
    if policy is None and isinstance(action_space, Box):
        policy = mlp_gaussian_policy
    elif policy is None and isinstance(action_space, Discrete):
        policy = mlp_categorical_policy

    with tf.variable_scope('pi'):
        policy_outs = policy(x, a, hidden_sizes, activation, output_activation, action_space)
        pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent = policy_outs

    with tf.variable_scope('vf'):
        v = tf.squeeze(mlp(x, list(hidden_sizes)+[1], activation, None), axis=1)

    with tf.variable_scope('vc'):
        vc = tf.squeeze(mlp(x, list(hidden_sizes)+[1], activation, None), axis=1)

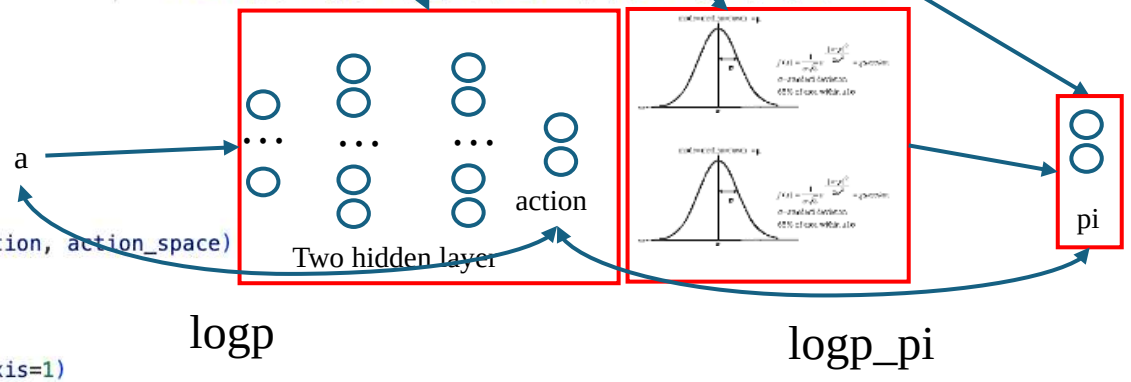
    return pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent, v, vc
```

```python
def mlp_gaussian_policy(x, a, hidden_sizes, activation, output_activation, action_space):
    act_dim = a.shape.as_list()[-1]
    mu = mlp(x, list(hidden_sizes)+[act_dim], activation, output_activation)
    log_std = tf.get_variable(name='log_std', initializer=-0.5*np.ones(act_dim, dtype=np.float32))
    std = tf.exp(log_std)
    pi = mu + tf.random_normal(tf.shape(mu)) * std
    logp = gaussian_likelihood(a, mu, log_std)
    logp_pi = gaussian_likelihood(pi, mu, log_std)

    old_mu_ph, old_log_std_ph = placeholders(act_dim, act_dim)
    d_kl = gaussian_kl(mu, log_std, old_mu_ph, old_log_std_ph)
    ent = gaussian_entropy(log_std)

    pi_info = {'mu': mu, 'log_std': log_std}
    pi_info_phs = {'mu': old_mu_ph, 'log_std': old_log_std_ph}

    return pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent
```



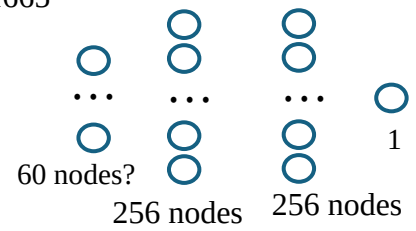

```python
def mlp(x, hidden_sizes=(32,), activation=tf.tanh, output_activation=None):
    for h in hidden_sizes[:-1]:
        x = tf.layers.dense(x, units=h, activation=activation)
    return tf.layers.dense(x, units=hidden_sizes[-1], activation=output_activation)
```

```python
def gaussian_likelihood(x, mu, log_std):
    pre_sum = -0.5 * (((x-mu)/(tf.exp(log_std)+EPS))**2 + 2*log_std + np.log(2*np.pi))
    return tf.reduce_sum(pre_sum, axis=1)
```

Run_agent.py

adv_ph: advantage function
cadv_ph: cost advantage function
ret_ph: return function
cret_ph: cost return function

60*256 + 256 + 256*256 + 256 + 256*1 + 1
=81665




```
with tf.variable_scope('pi'):
    policy_outs = policy(x, a, hidden_sizes, activation, output_activation, action_space)
    pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent = policy_outs

with tf.variable_scope('vf'):
    v = tf.squeeze(mlp(x, list(hidden_sizes)+[1], activation, None), axis=1)

with tf.variable_scope('vc'):
    vc = tf.squeeze(mlp(x, list(hidden_sizes)+[1], activation, None), axis=1)
```

```
Number of parameters:    pi: 262,        v: 81665,       vc: 81665
```

```
def get_vars(scope=''):
    return [x for x in tf.trainable_variables() if scope in x.name]

def count_vars(scope=''):
    v = get_vars(scope)
    return sum([np.prod(var.shape.as_list()) for var in v])
```

Trainable lines

```
#=========================================================================#
#  Create computation graph for actor and critic (not training routine)   #
#=========================================================================#

# Share information about action space with policy architecture
ac_kwargs['action_space'] = env.action_space

# Inputs to computation graph from environment spaces
x_ph, a_ph = placeholders_from_spaces(env.observation_space, env.action_space)

# Inputs to computation graph for batch data
adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph = placeholders(*(None for _ in range(5)))

# Inputs to computation graph for special purposes
surr_cost_rescale_ph = tf.placeholder(tf.float32, shape=())
cur_cost_ph = tf.placeholder(tf.float32, shape=())

# Outputs from actor critic
ac_outs = actor_critic(x_ph, a_ph, **ac_kwargs)
pi, logp, logp_pi, pi_info, pi_info_phs, d_kl, ent, v, vc = ac_outs

# Organize placeholders for zipping with data from buffer on updates
buf_phs = [x_ph, a_ph, adv_ph, cadv_ph, ret_ph, cret_ph, logp_old_ph]
buf_phs += values_as_sorted_list(pi_info_phs)

# Organize symbols we have to compute at each step of acting in env
get_action_ops = dict(pi=pi,
                      v=v,
                      logp_pi=logp_pi,
                      pi_info=pi_info)

# If agent is reward penalized, it doesn't use a separate value function
# for costs and we don't need to include it in get_action_ops; otherwise we do.
if not(agent.reward_penalized):
    get_action_ops['vc'] = vc

# Count variables
var_counts = tuple(count_vars(scope) for scope in ['pi', 'vf', 'vc'])
logger.log('\nNumber of parameters: \t pi: %d, \t v: %d, \t vc: %d\n'%var_counts)

# Make a sample estimate for entropy to use as sanity check
approx_ent = tf.reduce_mean(-logp)
```

Run_agent.py

```
113     #=========================================================================#
114     #  Create replay buffer                                                   #
115     #=========================================================================#
116
117     # Obs/act shapes
118     obs_shape = env.observation_space.shape
119     act_shape = env.action_space.shape
120
121     # Experience buffer
122     local_steps_per_epoch = int(steps_per_epoch / num_procs())
123     pi_info_shapes = {k: v.shape.as_list()[1:] for k,v in pi_info_phs.items()}
124     buf = CPOBuffer(local_steps_per_epoch,
125                     obs_shape,
126                     act_shape,
127                     pi_info_shapes,
128                     gamma,
129                     lam,
130                     cost_gamma,
131                     cost_lam)
```

Run_agent.py

```
#=====================================================================#
#  Create computation graph for policy learning                       #
#=====================================================================#

# Likelihood ratio
ratio = tf.exp(logp - logp_old_ph)

# Surrogate advantage / clipped surrogate advantage
if agent.clipped_adv:…
else:
    surr_adv = tf.reduce_mean(ratio * adv_ph)

# Surrogate cost
surr_cost = tf.reduce_mean(ratio * cadv_ph)

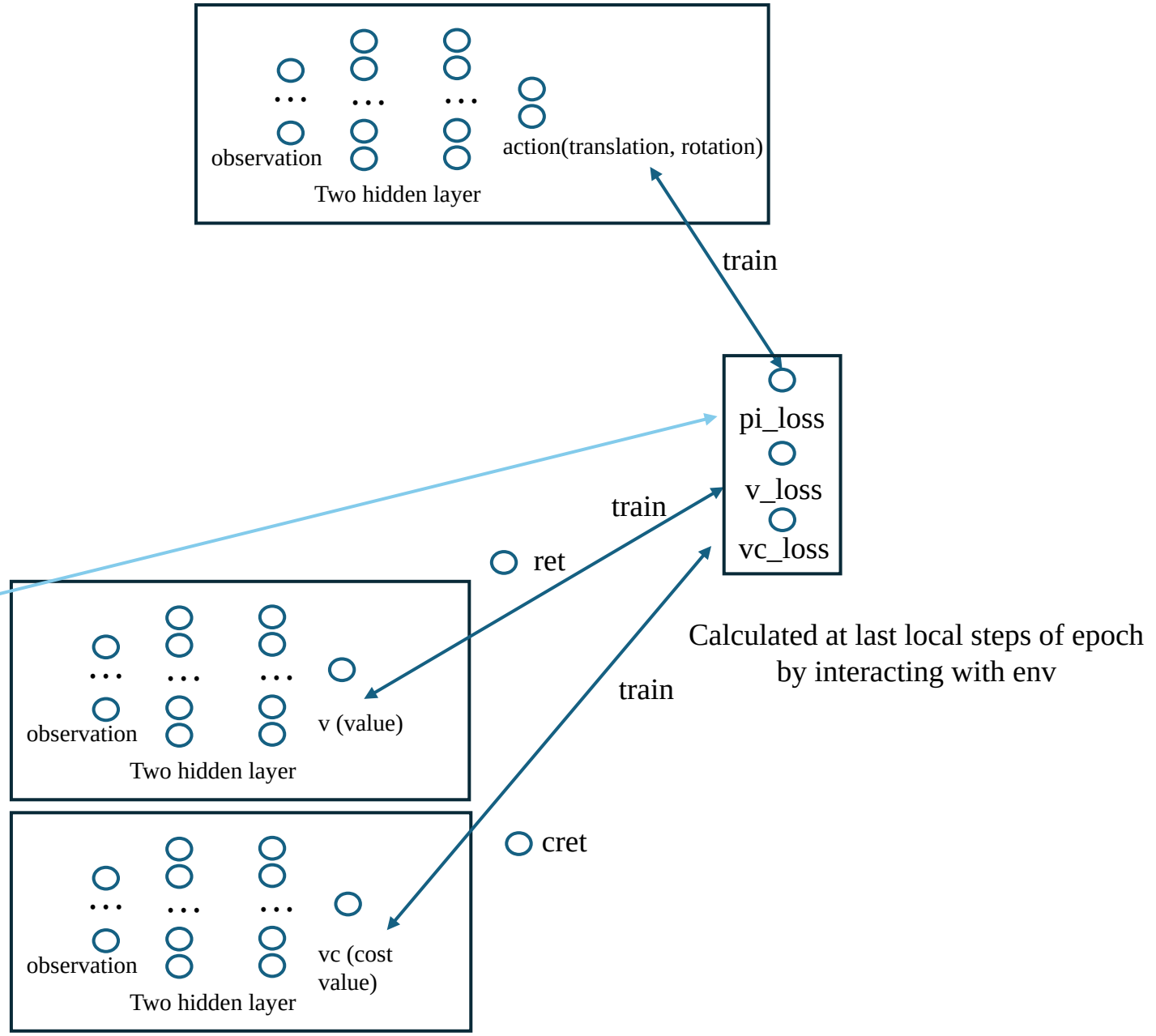
# Create policy objective function, including entropy regularization
pi_objective = surr_adv + ent_reg * ent

# Possibly include surr_cost in pi_objective
if agent.objective_penalized:…

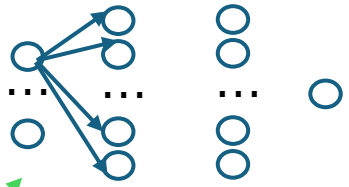
# Loss function for pi is negative of pi_objective
pi_loss = -pi_objective
```

Cpo doesn't use penalty

Detailed calculation of ratio, objective, loss functions are based on Trust Region Policy Optimization paper



Run_agent.py



get_pi_params

```
# Optimizer-specific symbols
if agent.trust_region:
    # Symbols needed for CG solver for any trust region method
    pi_params = get_vars('pi')
    flat_g = tro.flat_grad(pi_loss, pi_params)
    v_ph, hvp = tro.hessian_vector_product(d_kl, pi_params)
    if agent.damping_coeff > 0:
        hvp += agent.damping_coeff * v_ph

    # Symbols needed for CG solver for CPO only
    flat_b = tro.flat_grad(surr_cost, pi_params)

    # Symbols for getting and setting params
    get_pi_params = tro.flat_concat(pi_params)
    pdb.set_trace()
    set_pi_params = tro.assign_params_from_flat(v_ph, pi_params)

    training_package = dict(flat_g=flat_g,
                            flat_b=flat_b,
                            v_ph=v_ph,
                            hvp=hvp,
                            get_pi_params=get_pi_params,
                            set_pi_params=set_pi_params)
# Provide training package to agent
training_package.update(dict(pi_loss=pi_loss,
                             surr_cost=surr_cost,
                             d_kl=d_kl,
                             target_kl=target_kl,
                             cost_lim=cost_lim))
agent.prepare_update(training_package)
```

update pi_params according to gradients

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} \mathop{\mathrm{E}}_{\substack{s\sim d^{\pi_k} \\ a\sim\pi}} [A^{\pi_k}(s,a)]$$

$$\text{s.t. } J_{C_i}(\pi_k) + \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s\sim d^{\pi_k} \\ a\sim\pi}} \left[A^{\pi_k}_{C_i}(s,a)\right] \le d_i \quad \forall i$$

$$\bar{D}_{KL}(\pi || \pi_k) \le \delta. \tag{10}$$

For policies with high-dimensional parameter spaces like neural networks, (10) can be impractical to solve directly because of the computational cost. However, for small step sizes δ, the objective and cost constraints are well-approximated by linearizing around π_k, and the KL-divergence constraint is well-approximated by second order expansion (at $\pi_k = \pi$, the KL-divergence and its gradient are both zero). Denoting the gradient of the objective as g, the gradient of constraint i as b_i, the Hessian of the KL-divergence as H, and defining $c_i \doteq J_{C_i}(\pi_k) - d_i$, the

$$\theta_{k+1} = \arg\max_\theta g^T(\theta - \theta_k)$$

$$\text{s.t. } c_i + b_i^T(\theta - \theta_k) \le 0 \quad i = 1, ..., m$$

$$\frac{1}{2}(\theta - \theta_k)^T H(\theta - \theta_k) \le \delta. \tag{11}$$

Run_agent.py

```
#=====================================================================#
#  Create computation graph for value learning                        #
#=====================================================================#

# Value losses
v_loss = tf.reduce_mean((ret_ph - v)**2)
vc_loss = tf.reduce_mean((cret_ph - vc)**2)

# If agent uses penalty directly in reward function, don't train a separate
# value function for predicting cost returns. (Only use one vf for r - p*c.)
pdb.set_trace()
if agent.reward_penalized:
    total_value_loss = v_loss
else:
    total_value_loss = v_loss + vc_loss

# Optimizer for value learning
train_vf = MpiAdamOptimizer(learning_rate=vf_lr).minimize(total_value_loss)

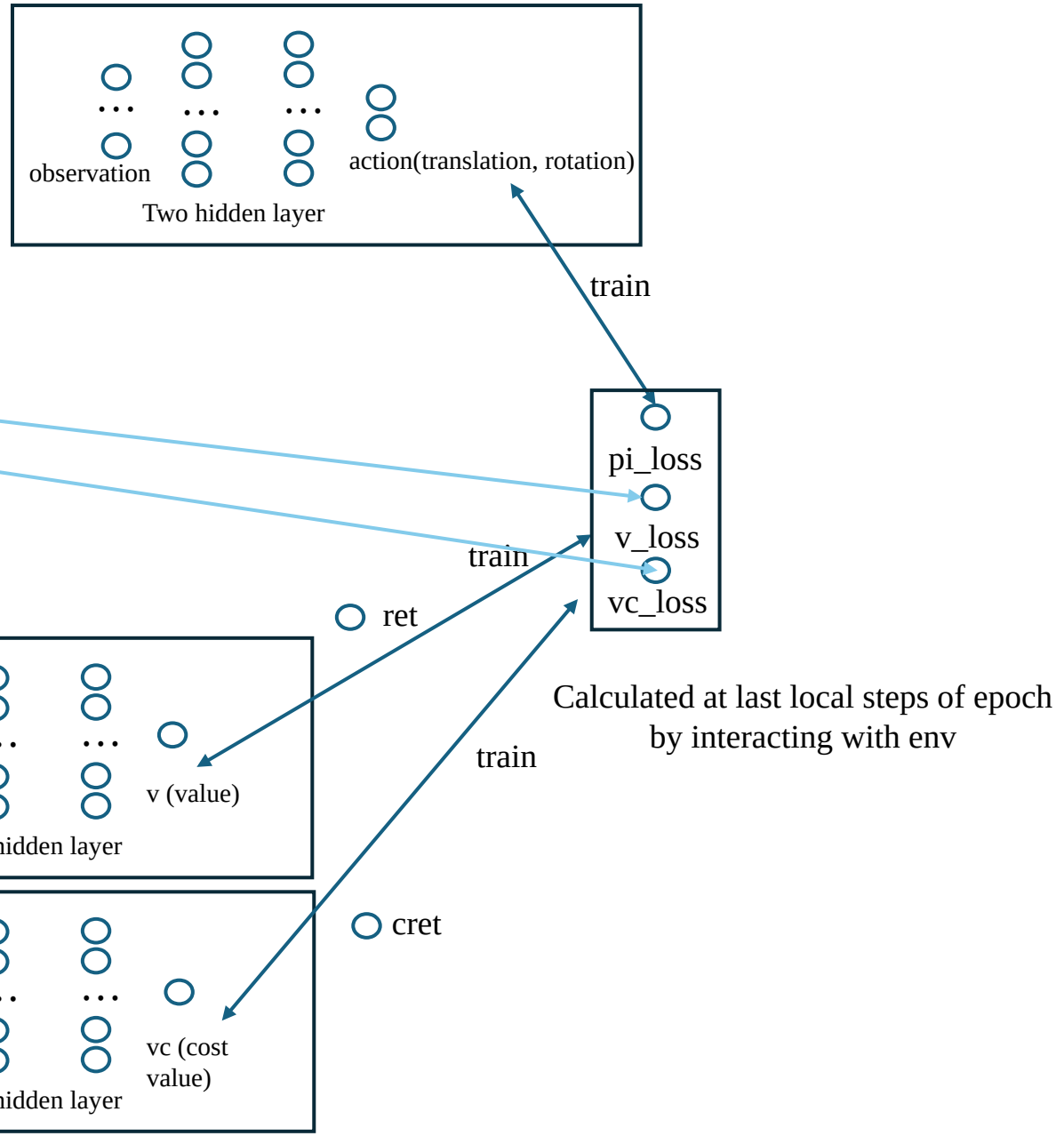
#=====================================================================#
#  Create session, sync across procs, and set up saver                #
#=====================================================================#

sess = tf.Session()
sess.run(tf.global_variables_initializer())

# Sync params across processes
sess.run(sync_all_params())

# Setup model saving
logger.setup_tf_saver(sess, inputs={'x': x_ph}, outputs={'pi': pi, 'v': v, 'vc': vc})


#=====================================================================#
#  Provide session to agent                                           #
#=====================================================================#
agent.prepare_session(sess)
```



Run_agent.py

```
#=====================================================================#
#  Create function for running update (called at end of each epoch)   #
#=====================================================================#

def update():
    cur_cost = logger.get_stats('EpCost')[0]
    c = cur_cost - cost_lim
    if c > 0 and agent.cares_about_cost:
        logger.log('Warning! Safety constraint is already violated.', 'red')

    #=====================================================================#
    #  Prepare feed dict                                                  #
    #=====================================================================#

    inputs = {k:v for k,v in zip(buf_phs, buf.get())}
    inputs[surr_cost_rescale_ph] = logger.get_stats('EpLen')[0]
    inputs[cur_cost_ph] = cur_cost

    #=====================================================================#
```

Constraint evaluation

```
    #=====================================================================#
    #  Make some measurements before updating                             #
    #=====================================================================#

    measures = dict(LossPi=pi_loss,
                    SurrCost=surr_cost,
                    LossV=v_loss,
                    Entropy=ent)
    if not(agent.reward_penalized):
        measures['LossVC'] = vc_loss
    if agent.use_penalty:
        measures['Penalty'] = penalty

    pre_update_measures = sess.run(measures, feed_dict=inputs)
    logger.store(**pre_update_measures)

    #=====================================================================#
    #  Update penalty if learning penalty                                 #
    #=====================================================================#
    if agent.learn_penalty:
        sess.run(train_penalty, feed_dict={cur_cost_ph: cur_cost})

    #=====================================================================#
    #  Update policy                                                      #
    #=====================================================================#
    agent.update_pi(inputs)

    #=====================================================================#
    #  Update value function                                              #
    #=====================================================================#
    for _ in range(vf_iters):
        sess.run(train_vf, feed_dict=inputs)
```

Update policy

```
    #=====================================================================#
    #  Make some measurements after updating                              #
    #=====================================================================#

    del measures['Entropy']
    measures['KL'] = d_kl

    post_update_measures = sess.run(measures, feed_dict=inputs)
    deltas = dict()
    for k in post_update_measures:
        if k in pre_update_measures:
            deltas['Delta'+k] = post_update_measures[k] - pre_update_measures[k]
    logger.store(KL=post_update_measures['KL'], **deltas)
```

agents.py

```
def update_pi(self, inputs):

    flat_g = self.training_package['flat_g']
    flat_b = self.training_package['flat_b']
    v_ph = self.training_package['v_ph']
    hvp = self.training_package['hvp']
    get_pi_params = self.training_package['get_pi_params']
    set_pi_params = self.training_package['set_pi_params']
    pi_loss = self.training_package['pi_loss']
    surr_cost = self.training_package['surr_cost']
    d_kl = self.training_package['d_kl']
    target_kl = self.training_package['target_kl']
    cost_lim = self.training_package['cost_lim']

    Hx = lambda x : mpi_avg(self.sess.run(hvp, feed_dict={**inputs, v_ph: x}))
    outs = self.sess.run([flat_g, flat_b, pi_loss, surr_cost], feed_dict=inputs)
    outs = [mpi_avg(out) for out in outs]
    g, b, pi_l_old, surr_cost_old = outs


    # Need old params, old policy cost gap (epcost - limit),
    # and surr_cost rescale factor (equal to average eplen).
    old_params = self.sess.run(get_pi_params)
    c = self.logger.get_stats('EpCost')[0] - cost_lim
    rescale = self.logger.get_stats('EpLen')[0]

    # Consider the right margin
    if self.learn_margin:
        self.margin += self.margin_lr * c
        self.margin = max(0, self.margin)

    # The margin should be the same across processes anyhow, but let's
    # mpi_avg it just to be 100% sure there's no drift. :)
    self.margin = mpi_avg(self.margin)
```

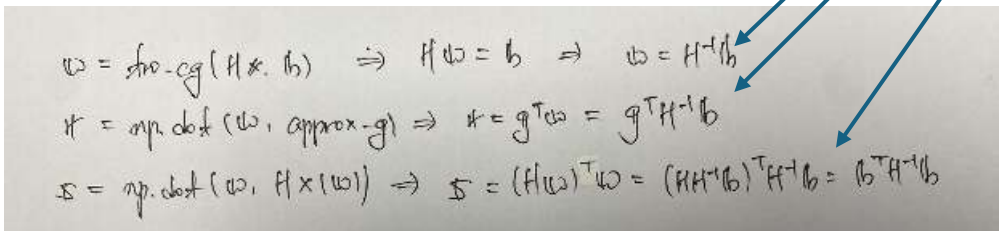
agents.py

$$\theta_{k+1} = \arg\max_{\theta} \; g^T(\theta - \theta_k)$$
$$\text{s.t.} \quad c_i + b_i^T(\theta - \theta_k) \leq 0 \quad i = 1, \ldots, m$$
$$\frac{1}{2}(\theta - \theta_k)^T H(\theta - \theta_k) \leq \delta. \tag{11}$$

$$\max_{\substack{\lambda \geq 0 \\ \nu \succeq 0}} \frac{-1}{2\lambda}\left(g^T H^{-1} g - 2r^T\nu + \nu^T S\nu\right) + \nu^T c - \frac{\lambda\delta}{2}, \tag{12}$$

g: gradient of the objective
b_i: gradient of constraint i
H: Hessian of the KL-divergence as H
$c_i := J_{Ci}(\pi_k) - d_i$

$$r \doteq g^T H^{-1} B, \; S \doteq B^T H^{-1} B.$$




```
# Adapt threshold with margin.
c += self.margin

# c + rescale * b^T (theta - theta_k) <= 0, equ
c /= (rescale + EPS)

# Core calculations for CPO
v = tro.cg(Hx, g)
approx_g = Hx(v)
q = np.dot(v, approx_g)

# Determine optim_case (switch condition for ca
# based on geometry of constrained optimization
if np.dot(b,b) <= 1e-8 and c < 0:
    # feasible and cost grad is zero---shortcut
    w, r, s, A, B = 0, 0, 0, 0, 0
    optim_case = 4
else:
    # cost grad is nonzero: CPO update!
    w = tro.cg(Hx, b)
    r = np.dot(w, approx_g)          # b^T H^{-1
    s = np.dot(w, Hx(w))             # b^T H^{-1
    A = q - r**2 / s                 # should be always positive (Cauchy-Shwarz)
    B = 2*target_kl - c**2 / s       # does safety boundary intersect trust region? (positive = yes)

    if c < 0 and B < 0:
        # point in trust region is feasible and safety boundary doesn't intersect
        # ==> entire trust region is feasible
        optim_case = 3
    elif c < 0 and B >= 0:
        # x = 0 is feasible and safety boundary intersects
        # ==> most of trust region is feasible
        optim_case = 2
    elif c >= 0 and B >= 0:
        # x = 0 is infeasible and safety boundary intersects
        # ==> part of trust region is feasible, recovery possible
        optim_case = 1
        self.logger.log('Alert! Attempting feasible recovery!', 'yellow')
    else:
        # x = 0 infeasible, and safety halfspace is outside trust region
        # ==> whole trust region is infeasible, try to fail gracefully
        optim_case = 0
        self.logger.log('Alert! Attempting infeasible recovery!', 'red')
```

```
# Surrogate cost
surr_cost = tf.reduce_mean(ratio * cadv_ph)

# Create policy objective function, including entropy regularization
pi_objective = surr_adv + ent_reg * ent

# Possibly include surr_cost in pi_objective
if agent.objective_penalized:
    pi_objective -= penalty * surr_cost
    pi_objective /= (1 + penalty)

# Loss function for pi is negative of pi_objective
pi_loss = -pi_objective

# Optimizer-specific symbols
if agent.trust_region:

    # Symbols needed for CG solver for any trust region method
    pi_params = get_vars('pi')
    flat_g = tro.flat_grad(pi_loss, pi_params)
    v_ph, hvp = tro.hessian_vector_product(d_kl, pi_params)
    if agent.damping_coeff > 0:
        hvp += agent.damping_coeff * v_ph

    # Symbols needed for CG solver for CPO only
    flat_b = tro.flat_grad(surr_cost, pi_params)
```

agents.py

```
if optim_case in [3,4]:
    lam = np.sqrt(q / (2*target_kl))
    nu = 0
elif optim_case in [1,2]:
    LA, LB = [0, r /c], [r/c, np.inf]
    LA, LB = (LA, LB) if c < 0 else (LB, LA)
    proj = lambda x, L : max(L[0], min(L[1], x))
    lam_a = proj(np.sqrt(A/B), LA)
    lam_b = proj(np.sqrt(q/(2*target_kl)), LB)
    f_a = lambda lam : -0.5 * (A / (lam+EPS) + B * lam) - r*c/(s+EPS)
    f_b = lambda lam : -0.5 * (q / (lam+EPS) + 2 * target_kl * lam)
    lam = lam_a if f_a(lam_a) >= f_b(lam_b) else lam_b
    nu = max(0, lam * c - r) / (s + EPS)
else:
    lam = 0
    nu = np.sqrt(2 * target_kl / (s+EPS))

# normal step if optim_case > 0, but for optim_case =0,
# perform infeasible recovery: step to purely decrease cost
x = (1./(lam+EPS)) * (v + nu * w) if optim_case > 0 else nu * w

# save intermediates for diagnostic purposes
self.logger.store(Optim_A=A, Optim_B=B, Optim_c=c,
                  Optim_q=q, Optim_r=r, Optim_s=s,
                  Optim_Lam=lam, Optim_Nu=nu,
                  Penalty=nu, DeltaPenalty=0,
                  Margin=self.margin,
                  OptimCase=optim_case)

def set_and_eval(step):
    self.sess.run(set_pi_params, feed_dict={v_ph: old_params - step * x})
    return mpi_avg(self.sess.run([d_kl, pi_loss, surr_cost], feed_dict=inputs))
```

Update neural network(pi parameters)

$$\theta_{k+1} = \arg\max_{\theta} \; g^T(\theta - \theta_k)$$
$$\text{s.t.} \quad c_i + b_i^T(\theta - \theta_k) \le 0 \quad i = 1, \ldots, m$$
$$\frac{1}{2}(\theta - \theta_k)^T H(\theta - \theta_k) \le \delta. \tag{11}$$

$$\max_{\substack{\lambda \ge 0 \\ \nu \succeq 0}} \frac{-1}{2\lambda}\left(g^T H^{-1} g - 2r^T\nu + \nu^T S\nu\right) + \nu^T c - \frac{\lambda\delta}{2}, \tag{12}$$

g: gradient of the objective
b_i: gradient of constraint i
H: Hessian of the KL-divergence as H
$c_i := J_{Ci}(\pi_k) - d_i$

$$r \doteq g^T H^{-1} B, \; S \doteq B^T H^{-1} B.$$

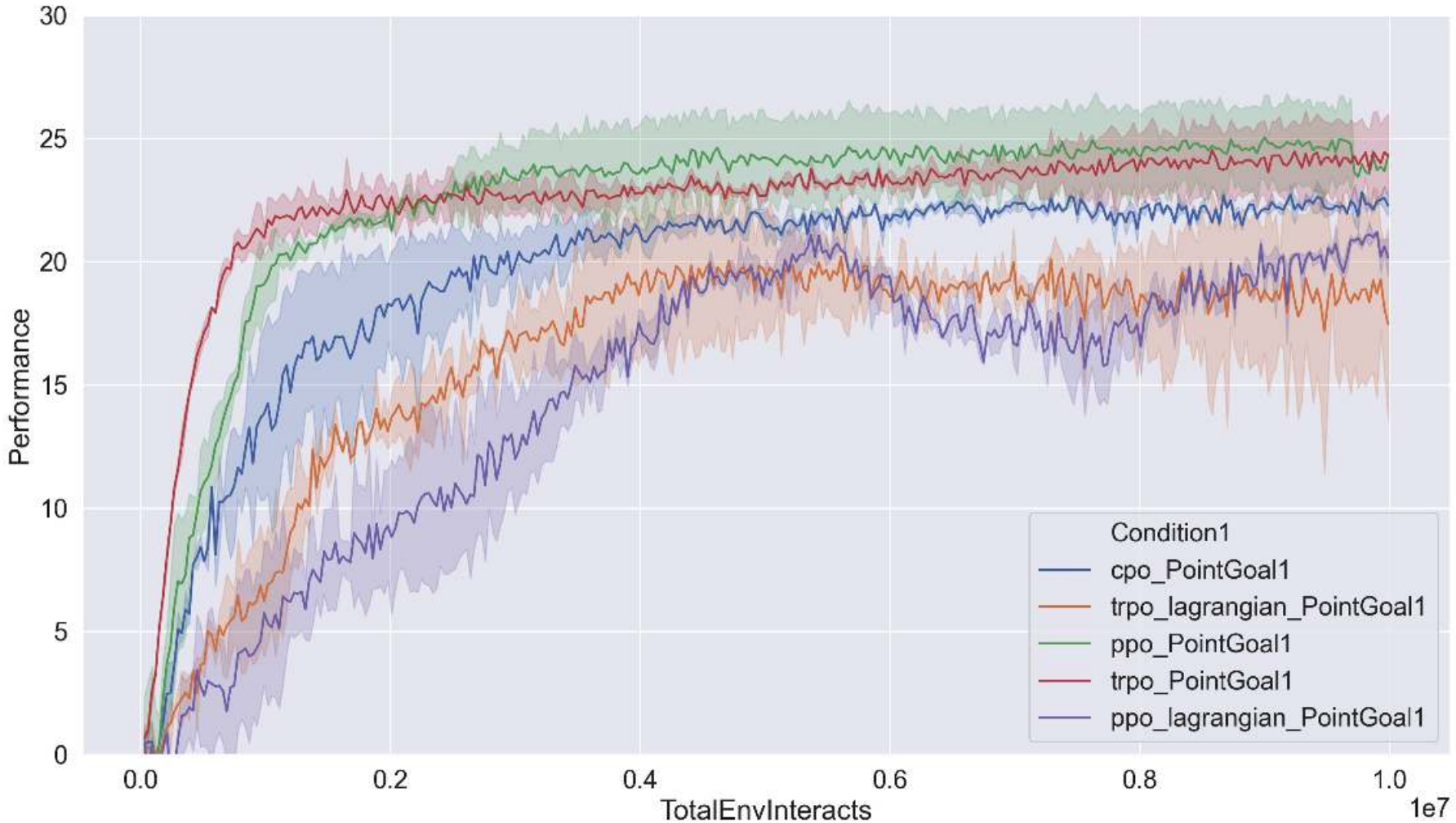
$$\theta^* = \theta_k + \frac{1}{\lambda^*} H^{-1}\left(g - B\nu^*\right)$$

Future works

1. Just one constraint
   - More constraint?
   - Detailed constraint?
2. Different cost function rather than cost by time?
3. What if being in the unsafe region is strongly forbiddened?
4. Neural network
   - Network size
   - Network depth

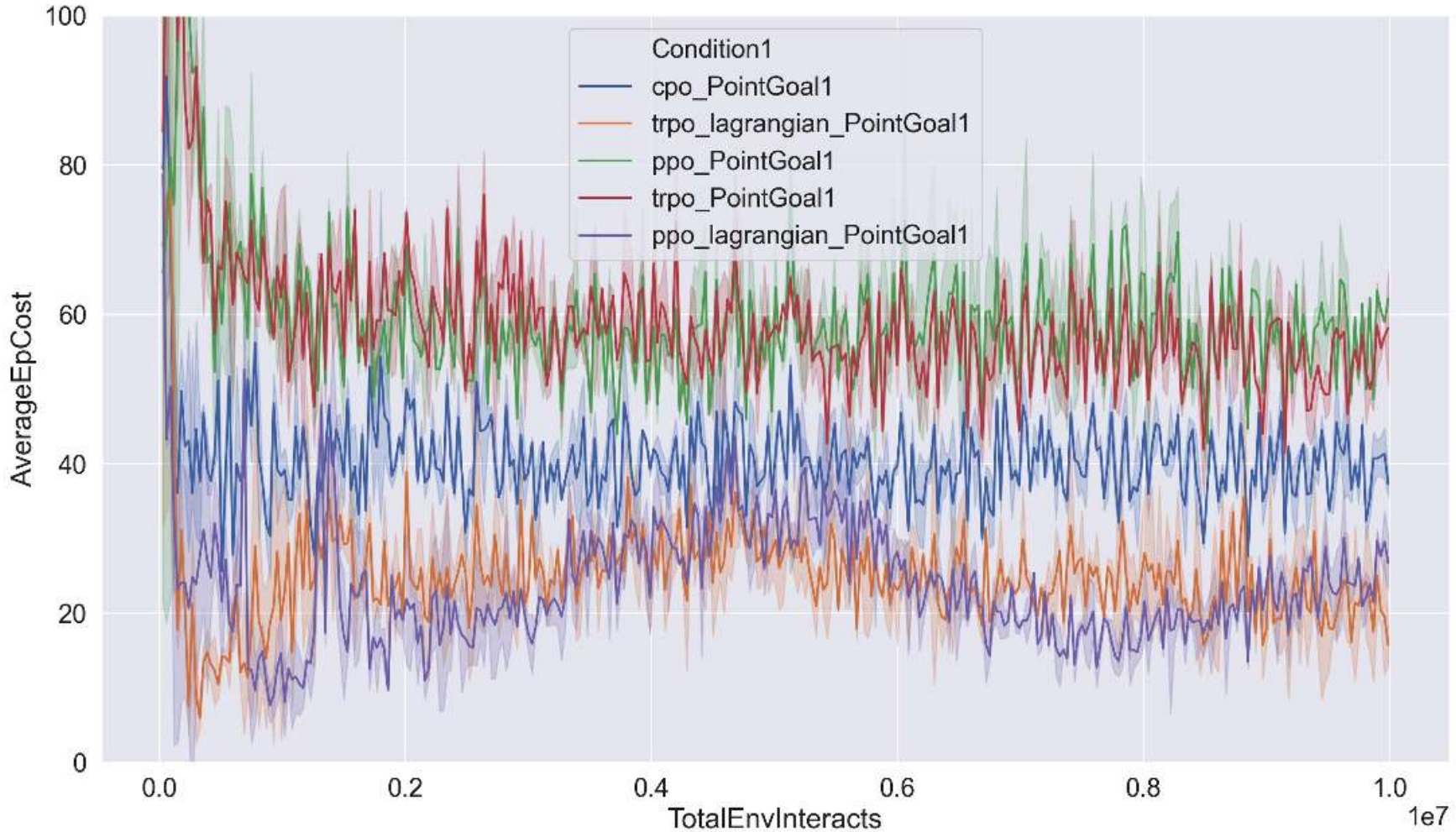
Results

5 algorithms: ppo, ppo_lagrangian, trpo, trpo_lagrangian, cpo



Results

5 algorithms: ppo, ppo_lagrangian, trpo, trpo_lagrangian, cpo

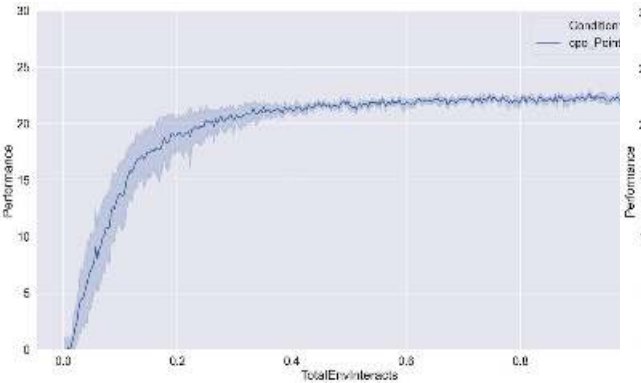


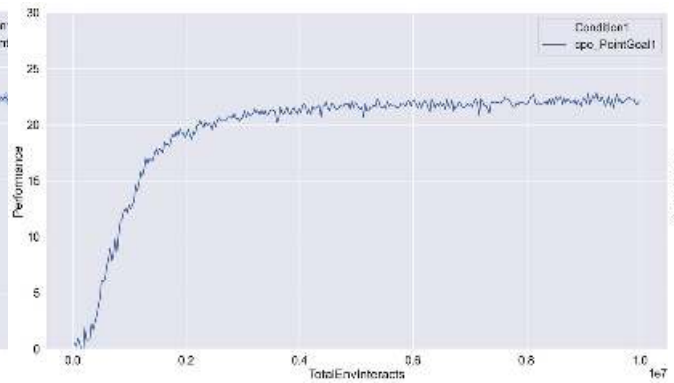
Results

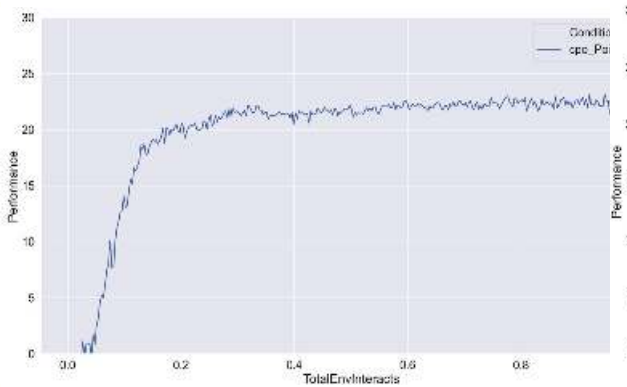
Hidden layer size was (64, 32) in the paper, but in code, the default value was (256, 256)
So, I tried different hidden layer size.

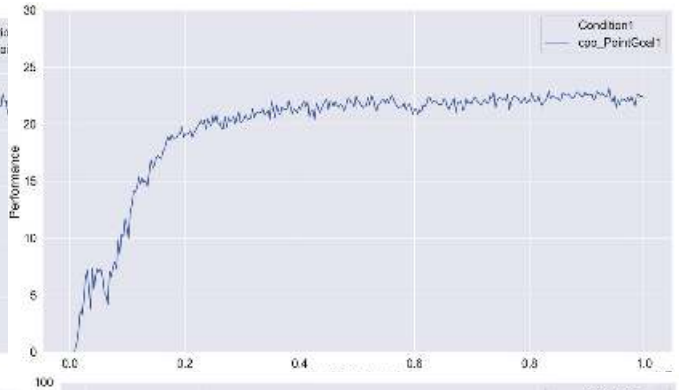
There are three mlp (policy, value, cost value).
Firstly, I tested with same hidden layer size for three mlp.

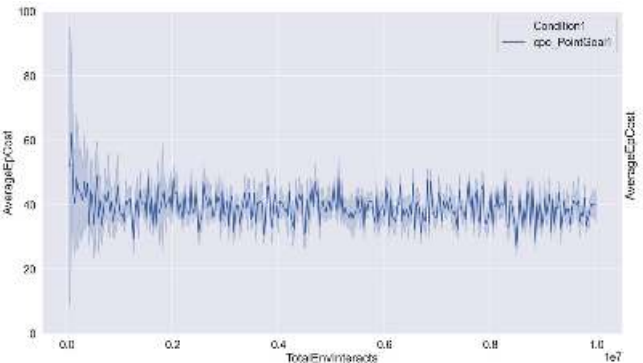
(256, 256) (64, 32) (32, 16) (4,2)

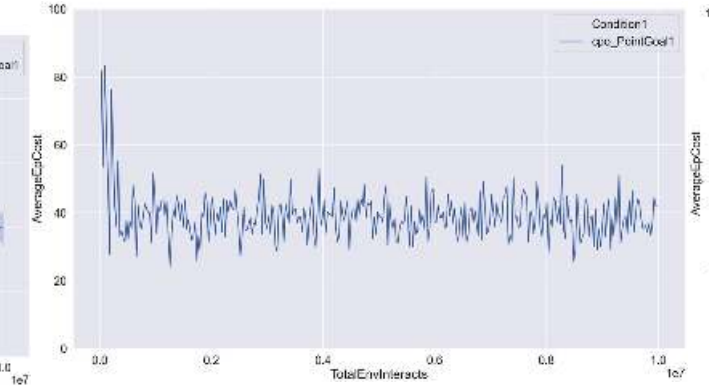


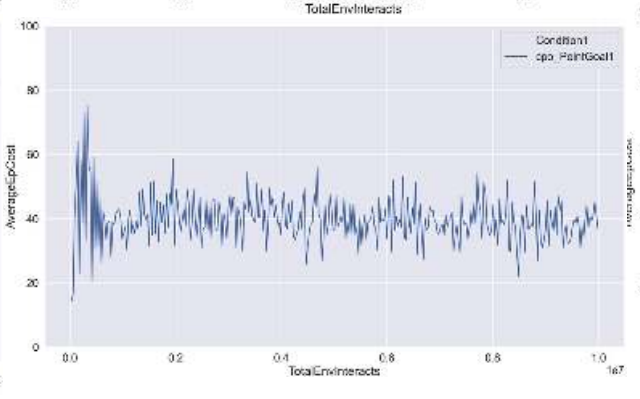


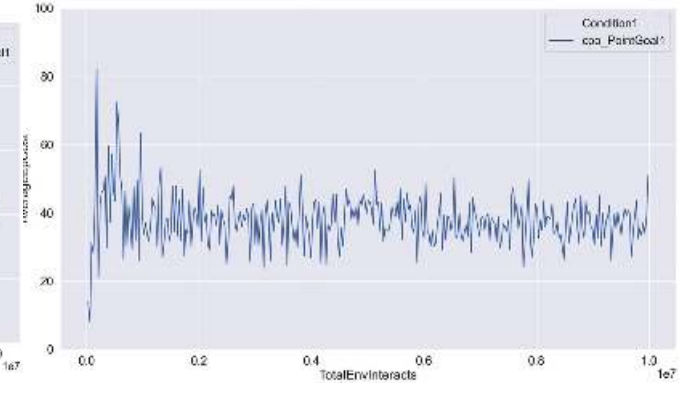










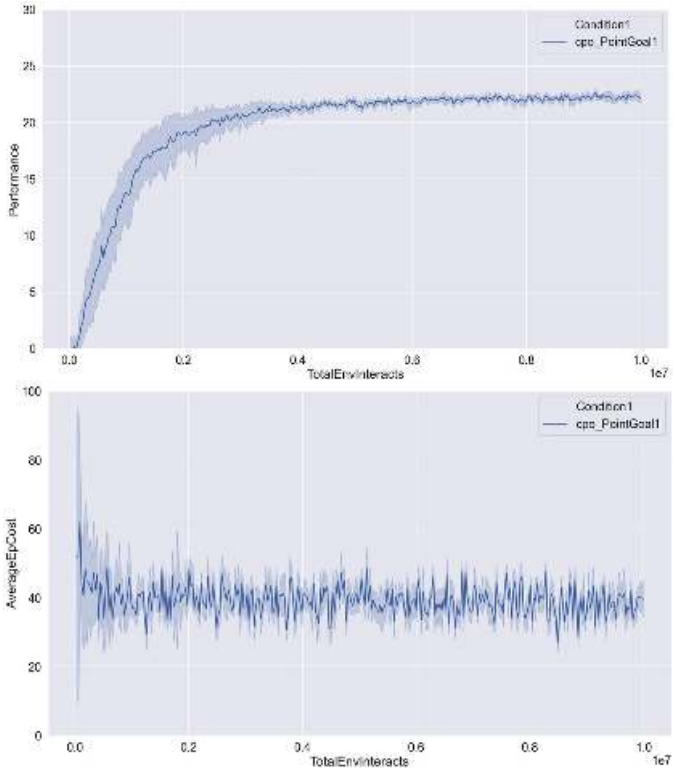


Results

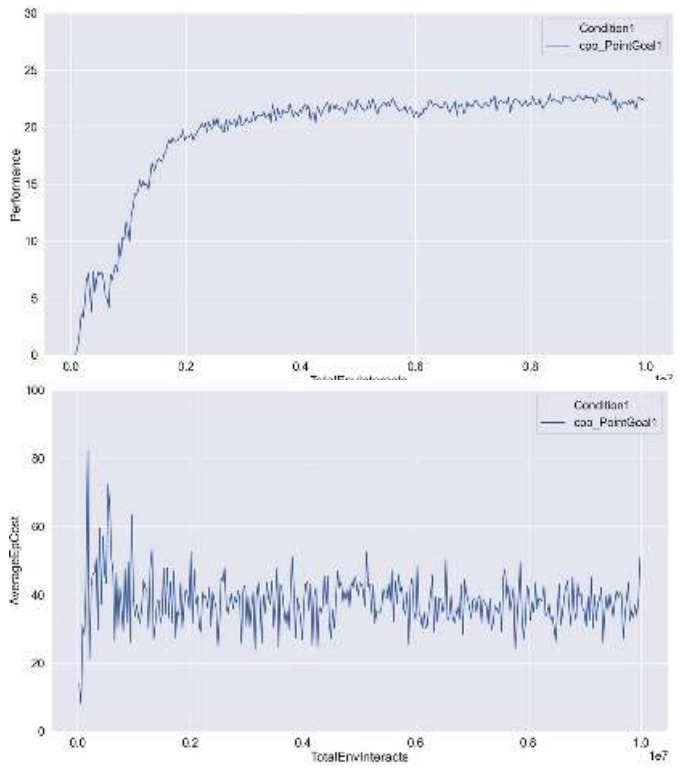
Is it just enough with (4, 2) hidden layer size?
Or, value and cost value function is miscalculated due to small hidden layer size.

So, I run one more simulation: hidden layer size for policy is (4, 2) / hidden layer size for value, cost value function are (256, 256)

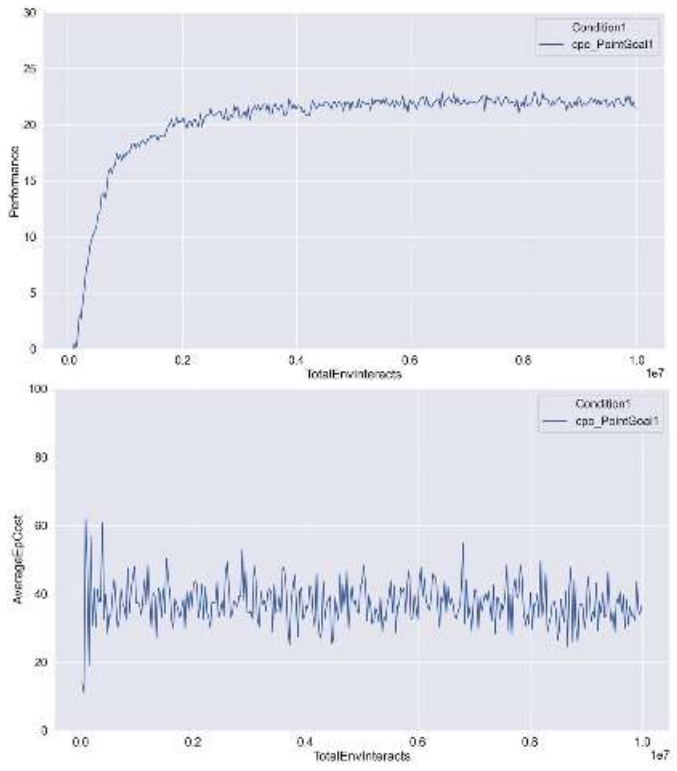
(256, 256)



(4,2)



hidden layer size for policy is (4, 2)
hidden layer size for value, cost value function are (256, 256)

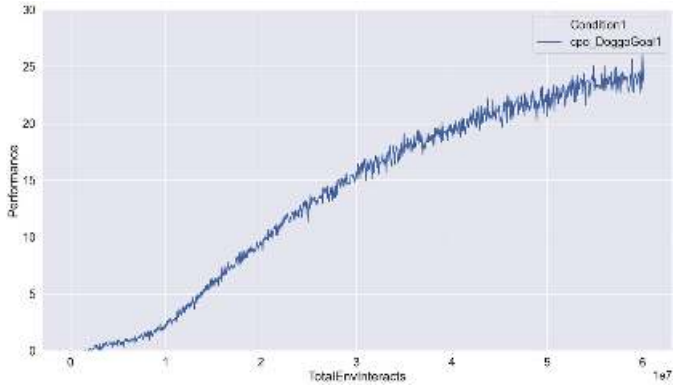


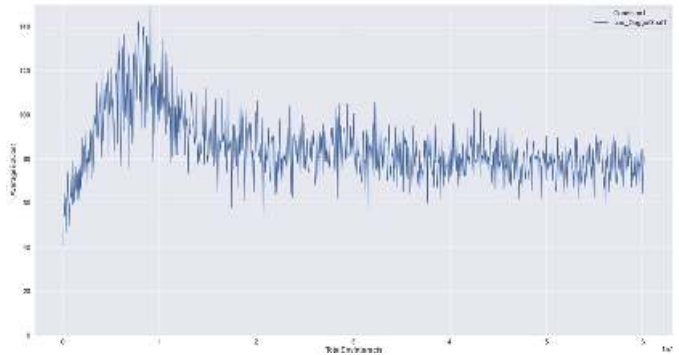
Results

I seems that small hidden layer size is enough for training.
Why? Because action space is dimension of 2?

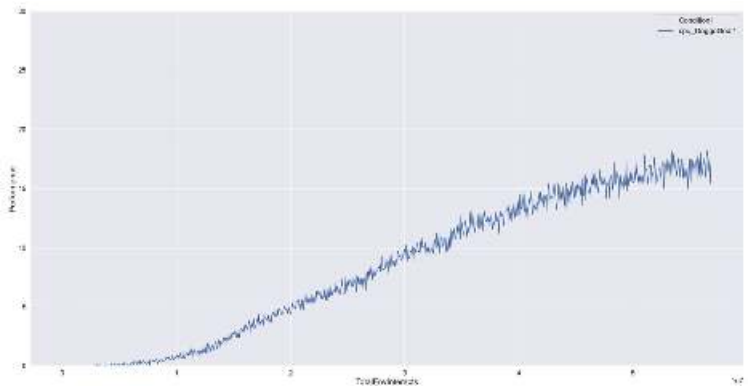
So, I conducted simulation with robot ”Doggo”: hidden layer size (256, 256) / (32, 32)

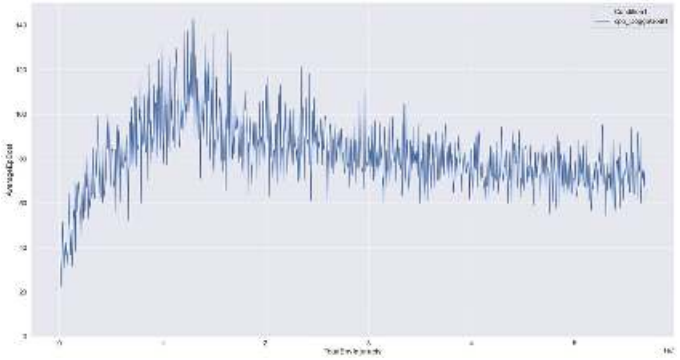
(256, 256)





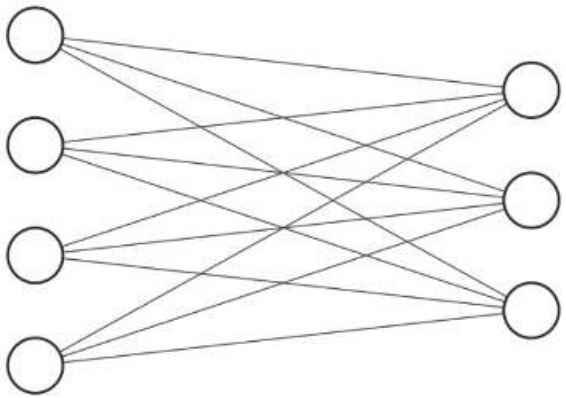
(32, 32)





Appendix

Appendix - Neural network



Input Layer $\in \mathbb{R}^4$ Output Layer $\in \mathbb{R}^3$

```
입력값 = [2.3, 5.6, 1.2, 2.4]
가중치1 = [3.1, -2.1, 8.7, -9.1]
가중치2 = [4.5, 2.1, -1.7, 3.4]
가중치3 = [2.3, -6.4, 2.1, -6.3]
편향1 = 2
편향2 = 3
편향3 = 1

출력값1 = 입력값[0]*가중치1[0] + 입력값[1]*가중치1[1] + 입력값[2]*가중치1[2] + 입력값[3]*가중치1[3]\
+ 편향1
출력값2 = 입력값[0]*가중치2[0] + 입력값[1]*가중치2[1] + 입력값[2]*가중치2[2] + 입력값[3]*가중치2[3]\
+ 편향2
출력값3 = 입력값[0]*가중치3[0] + 입력값[1]*가중치3[1] + 입력값[2]*가중치3[2] + 입력값[3]*가중치3[3]\
+ 편향3

출력값 = [출력값1, 출력값2, 출력값3]

print(출력값)
```

```
입력값 = [2.3, 5.6, 1.2, 2.4]
가중치 = [[3.1, -2.1, 8.7, -9.1],
          [4.5, 2.1, -1.7, 3.4],
          [2.3, -6.4, 2.1, -6.3]]
편향 = [2, 3, 1]
```

```
뉴런층출력값= []
for 뉴런가중치, 뉴런편향 in zip(가중치, 편향):
        뉴런출력값 = 0
    for n_입력값, 가중치 in zip(입력값, 뉴런가중치):
        뉴런출력값 += n_입력값*가중치
    뉴런출력값 += 편향
    뉴런층출력값.append()

print(뉴런층출력값)
```

```
출력값 = np.dot(가중치, 입력값) + 편향
print(출력값)
```

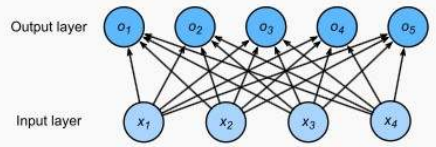
https://nobsai.tistory.com/8

Appendix - Multilayer Perceptron

3.8.1. 은닉층(hidden layer)

이전의 것을 기억해보면, 단일 선형 변환 (single linear transformation)을 통해서 입력들을 바로 출력으로 매핑을 했고, 이는 아래과 같이 표현할 수 있습니다.

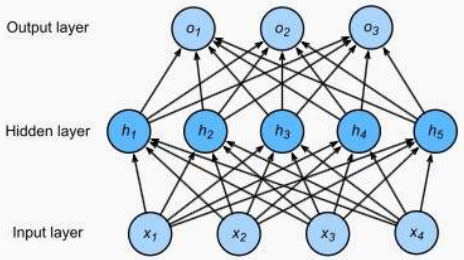
$$\hat{\mathbf{o}} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$$



3.8.2. 하나에서 여러개로

검정색이나 희색 이미지을 이용해서 강아지나 고양이를 분류하는 케이스를 생각해봅시다. 각 픽셀의 값을 증가시키면 강아지라고 판별할 확률값을 높이거나 내려가는 경우를 생각해봅시다. 이는 합당하지 않습니다. 왜냐하면, 이렇게 된다면 결국 강아지는 모두 검정색이고 고양이는 모두 흰색이거나 그 반대라는 것을 의미하기 때문입니다.

이미지에 무엇이 있는지 알아내기 위해서는 입력과 출력 간의 매우 복잡한 관계와 이를 위해서는 패턴이 많은 특성(feature)들 사이의 관계를 통해서 특정 지어지는 가능성을 고려해야합니다. 이런 경우에는, 선형 모델의 정확도는 낮을 것입니다. 우리는 한 개 이상의 은닉층(hidden layer)을 함께 사용해서 더 일반적인 함수들을 이용한 모델을 만들 수 있습니다. 이를 구현하는 가장 쉬운 방법은 각 층 위에 다른 층들을 쌓는 것입니다. 각 층의 결과는 그 위의 층의 입력으로 연결되는데, 이는 마지막 출력층까지 반복됩니다. 이런 아키텍쳐는 일반적으로 "다층 퍼셉트론(multilayer perceptron)"이라고 불립니다. 즉, MLP는 여러 층를 연속해서 쌓아올립니다. 예를 들면 다음과 같습니다.



위 다층 퍼셉트론(multilayer perceptron)에서는 입력이 4개, 출력이 3개, 중간의 은닉층(hidden layer)은 5개의 은닉 유닛(hidden unit)이 있습니다. 입력층은 어떤 연산을 수행하지 않기 때문에, 이 다층 퍼셉트론(multilayer perceptron)은 총 2개의 층(layer)을 갖습니다. 은닉층(hidden layer)의 뉴런(neuron)들은 입력층의 입력들과 모두 연결되어 있습니다. 출력층(output layer)의 뉴런과 은닉층의 뉴런들도 모두 연결되어 있습니다. 따라서, 이 다층 퍼셉트론의 은닉층과 출력층은 모두 완전 연결층(fully connected layer)입니다.

https://ko.d2l.ai/chapter_deep-learning-basics/mlp.html

Appendix – activation function

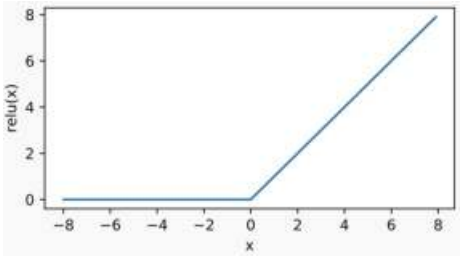
3.8.5. 활성화 함수(activation function)

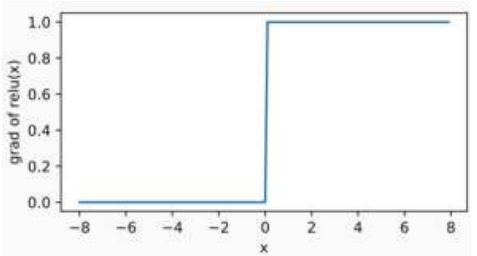
활성화 함수(activation function)의 예를 더 알아보겠습니다. 결국 딥 네트워트를 동작시키는 것은 선형(linear) 항목과 비선형(nonlinear) 항목들을 서로 교차 시키는 것입니다. 구현하기 간단하고 좋은 효과로 유명한 ReLU 함수가 있습니다.

3.8.6. ReLU 함수

ReLU (rectified linear unit) 함수는 아주 간단한 비선형 변환입니다. 주어진 x 에 대해서, ReLU 함수는 다음과 같이 정의됩니다.

$$\mathrm{ReLU}(x) = \max(x, 0).$$



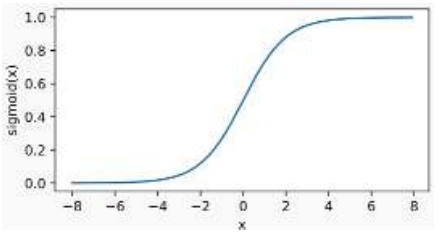


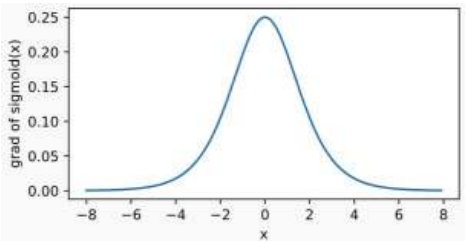
3.8.7. Sigmoid 함수

sigmoid 함수는 실수 값을 (0,1) 사이의 값으로 변환해줍니다.

$$\mathrm{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

$$\frac{d}{dx}\mathrm{sigmoid}(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2} = \mathrm{sigmoid}(x)\,(1 - \mathrm{sigmoid}(x))$$



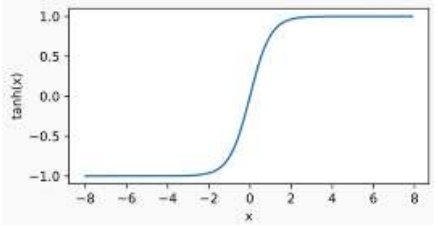


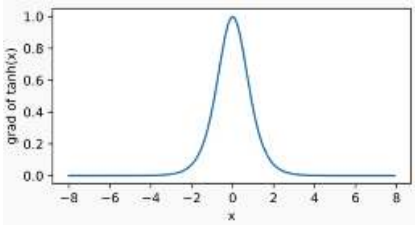
3.8.8. Tanh 함수

Tanh (Hyperbolic Tangent) 함수는 값을 -1와 1사이 값으로 변환합니다.

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x).$$



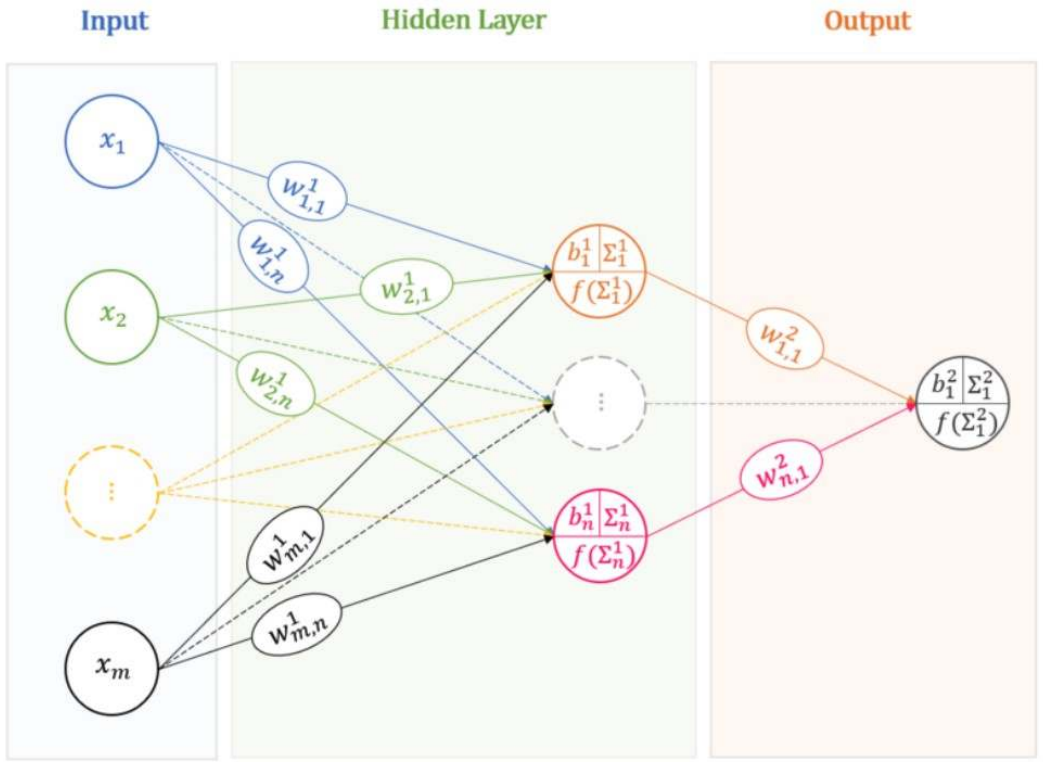


https://ko.d2l.ai/chapter_deep-learning-basics/mlp.html

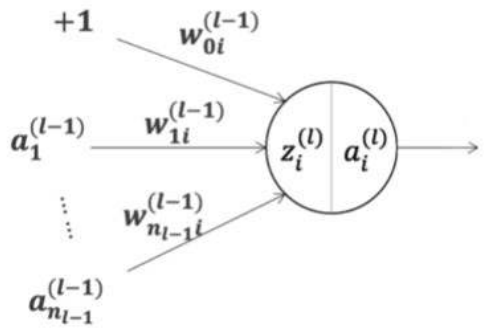
Appendix – Multilayer Perceptron

MLP (다층 퍼셉트론, Multi-layer Perceptron)

위에서 언급한 XOR 문제를 해결할 수 있는 수단으로서 다층 퍼셉트론(MLP)과 같은 심층 신경망이 등장하였다.
MLP는 퍼셉트론의 개념에서 은닉층을 추가하여 다양한 종류의 복잡한 패턴과 특징을 인식이 가능해졌다.



다층 퍼셉트론의 표현



입력 벡터 : $\mathbf{x} = [x_1, x_2, \ldots, x_m]$

가중치 행렬 : $\mathbf{W}^{(l)}$

→ l은 Layer을 의미한다.

가중치 : 각각의 가중치는 $w^{(l-1)}_{ji}$로 나타낼 수 있다.

→ i는 다음 Layer의 i번째 노드를 의미한다.

→ j는 이전 Layer의 j번째 노드를 의미한다.

→ 즉 이전 Layer의 j번째 노드에서 다음 Layer의 i번째 노드로의 가중치를 의미한다.

l번째 은닉층의 i번째 노드의 입력값

→ $z^{(l)}_i = \sum_{j=0}^{n_{l-1}} w^{(l-1)}_{ji} a^{(l-1)}_j$

l번째 은닉층의 i번째 노드의 출력값

→ $a^{(l)}_i = \sigma(z^{(l)}_i)$

→ σ는 활성 함수이다.

https://sjh9708.tistory.com/225

Appendix - Loss functions

우리는 삶 속에서 늘 '이것'을 교환합니다. 인간이라면 누구든 쓰는 언어가 이것의 좋은 매체입니다. 바로 정보입니다. 우리가 질량이나 높이, 속도와 같은 것들을 단위량을 갖고 측정하고 비교하듯이 정보 또한 그럴 수 있습니다. 그때 등장하는 개념이 **정보량**입니다.

이 정보량의 기원은 어떤 내용을 표현하기 위해 물어야 하는 최소한의 질문 개수에서 출발합니다. 예를 들어 0과 1만 구분하는 전기 신호로 동전을 5번 던진 결과를 전송해야 한다고 가정해보겠습니다. 우리가 물을 수 있는 질문은

$$\text{질문개수} = \log_2(\text{가능한 결과의 수})$$

이런 내용을 처음 정립한 것이 R.V.L Hartley입니다. 그는 논문에서 정보를 H로 표현합니다.

$$\begin{aligned} H &= n\log(s) \\ &= \log(s^n) \end{aligned}$$

n은 지금 보내는 문자열의 문자(글자, 숫자 등등) 개수, s는 각 선택에서 가능한 결과의 가지수입니다(알파벳은 26가지가 가능해서 아까 $log_2(26)$이었습니다). 한 마디로 말하면 1개의 문자를 구분해내기 위한 질문 개수(log_2(**가능한 결과의 수**)) 곱하기 몇 개의 문자가 왔는지입니다. 그러면 전체적으로 몇 번의 질문으로 해당 문자열을 파악할 수 있는지가

Entropy

[Information entropy 발췌]

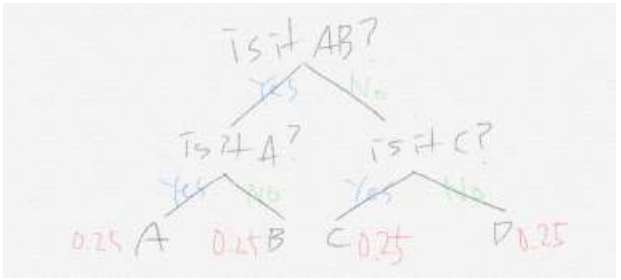
예를 들어 문자열을 출력하는 2개의 기계 '**X**'와 '**Y**'가 있다고 가정해보겠습니다.

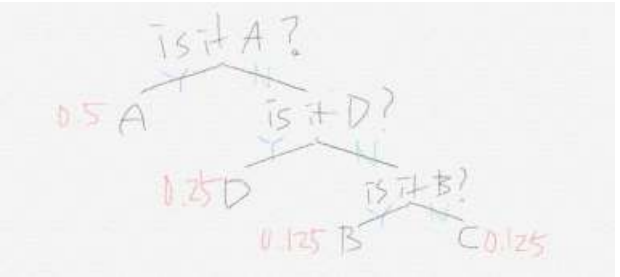
기계 X는

- A, B, C, D를 각각 0.25의 확률로 출력

반면, **기계 Y**는

- A : 0.5
- B: 0.125
- C: 0.125
- D: 0.25





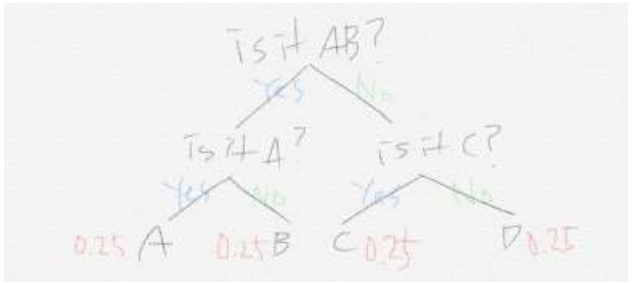
기계 Y의 경우에는 굳이 (A,B)중에 있니 (C,D)중에 있니라고 묻는 전략을 쓰는 것은 비효율적입니다. 왜냐하면 이미 A가 50%의 확률로 나타나기 때문입니다. 그러므로 처음에 물을 때 (A)니 아니면 (B,C,D)중에 있니 라고 묻는 게 옳습니다. 그리고 나서 또 (B,C,D)중에서는 0.5의 확률 중에서 D가 0.25를 차지하고 있으므로 (D)니 아니면 (B,C) 중에 있니 라고 물으면 됩니다. 그렇다면 기계 Y의 경우에 필요한 최소의 질문 개수는 앞서 보였던

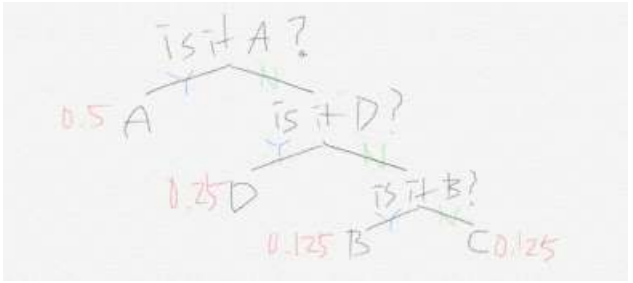
단순 로그식으로는 계산이 되지 않습니다. 그 대신 A가 나타날 확률에 A를 추려내기 위한 질문 개수(처음 1개의 질문만으로 A를 추려냅니다)를 곱합니다. 마찬가지로 B가 나타날 확률에 B를 추려내기 위한 질문 개수(3번의 질문으로 추려냅니다)를 곱하고 나머지 C,D에 대해서도 똑같이 하면 됩니다. 이를 식으로 나타내보면 이렇습니다.

$$p(A)\cdot 1 + p(B)\cdot 3 + p(C)\cdot 3 + p(D)\cdot 2 = 1.75$$

https://hyunw.kim/blog/2017/10/14/Entropy.html

Appendix - Loss functions





그러므로 우리는 1개의 글자를 기계 Y로부터 추려내기 위해서는 평균적으로 1.75개의 질문을 하면 됩니다. 만약 우리가 각 기계가 출력한 100개의 글자를 맞히기 위해서는 기계 X는 200번, 기계 Y는 175번의 질문을 해야 합니다. 그러므로 기계 Y가 X보다 더 적은 정보량을 생산한다고 볼 수 있습니다! 왜냐하면 불확실성(랜덤성)이 더 적기 때문입니다(모든 사건이

생각해보면 질문횟수 A는 1번, B,C는 3번, D는 2번은 A,B,C,D의 발생 확률과 연관이 있습니다.

- A : 0.5 = 1/2
- B: 0.125 = 1/8
- C: 0.125 = 1/8
- D: 0.25 =1/4

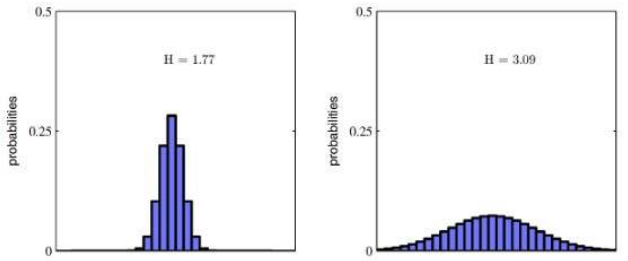
$$\log_2(\frac{1}{1/2}) = 1$$
$$\log_2(\frac{1}{1/8}) = 3$$
$$\log_2(\frac{1}{1/4}) = 2$$

Claude Shannon 의 Entropy

$$H = \sum(\text{사건 발생확률}) \cdot \log_2(\frac{1}{\text{사건 발생확률}})$$
$$= \sum_i p_i \log_2(\frac{1}{p_i})$$
$$= -\sum_i p_i \log_2(p_i)$$

질문 횟수

Entropy는 가능한 모든 사건이 같은 확률로 일어날 때 그 최댓값을 갖습니다. 그 각각의 확률이 모두 동등한 상황에서 조금만 벗어나도 entropy는 감소합니다. 아래 두 경우를 생각해보겠습니다. X의 범위에 걸쳐 특정 값들만 높은 왼쪽 분포가 값들끼리 서로 비슷비슷한 오른쪽 분포보다 entropy가 작습니다.

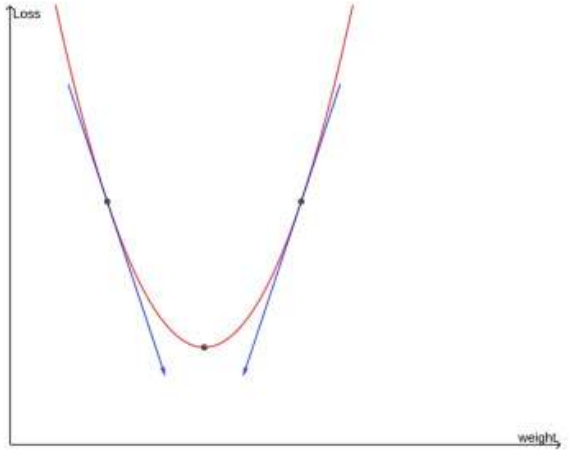


https://hyunw.kim/blog/2017/10/14/Entropy.html

Appendix - MLP – How to train?

가중치에 대한 차원

경사하강법은 기울기를 이용하는 방법이라고 했습니다. 두 점의 기울기를 살펴보겠습니다.



규칙이 보이네요. w값에서의 손실 함수의 미분계수가 음이라면 w를 양의 방향으로, 손실 함수의 미분계수가 양이라면 w를 음의 방향으로 이동시켜 주어야 합니다.

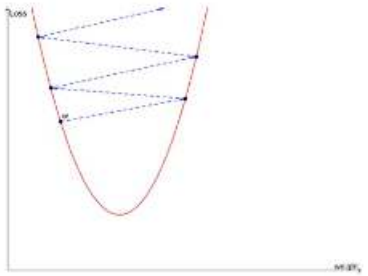
이제 w를 얼마만큼 이동시키는지가 관건이겠네요. 경사하강법에서는 다음과 같은 식으로 w값을 조정해줍니다.

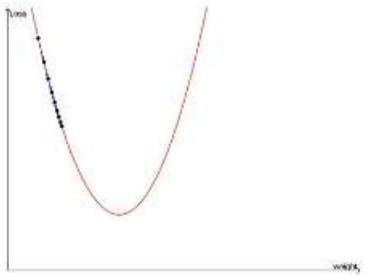
$$w = w - \alpha \times \frac{\partial L}{\partial w}$$

(w : weight, \(\alpha \) : learning-rate)

\(\alpha \)는 **학습률(Learning rate)**이라는 파라미터로, w값이 움직이는 거리를 조절해주기 위해 사용합니다.

만약 w가 이동하는 거리가 너무 크다면 손실 값이 커지는 방향으로 w가 조정될 가능성이 있고, 반대로 w가 이동하는 거리가 너무 작다면 w가 적합한 값으로 수렴하는데에 너무 오랜 시간이 걸립니다.





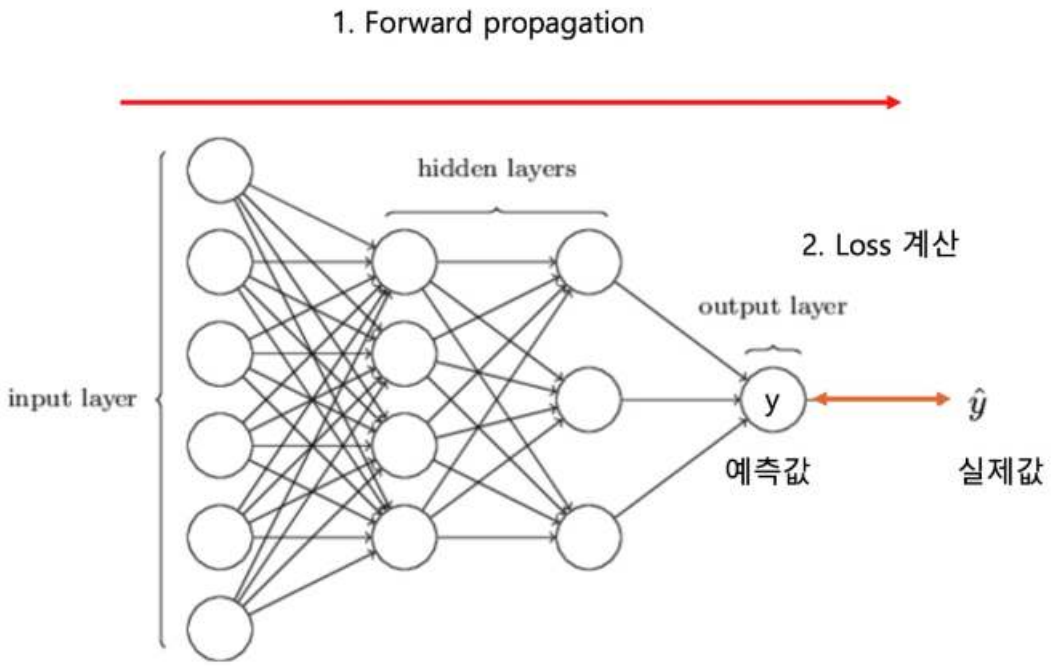
https://yhyun225.tistory.com/5
https://en.wikipedia.org/wiki/Loss_function

Appendix - MLP – How to train?

2. Loss 계산

Loss는 예측한 결과와 실제 결과간의 차이를 나타내주는 지표이다. 기계학습의 목표는 학습을 통해서 Loss를 최소화하는 것이다.

실제값과 모델의 예측값 사이의 차이를 나타내는 손실 함수(MSE, ☒Cross Entropy Loss 등)를 사용하여 계산한다.



출력의 개수와 손실 함수

Foward Propagation은 입력 벡터로부터 예측값을 도출하는 과정이다.

출력의 개수는 예측해야 하는 값의 차원을 나타낸다. **즉 예측해야 하는 Class(Label)의 수 만큼의 출력 뉴련이 필요하다.**

- **1개의 출력**: 일반적으로 **맞다/틀리다의 이진 분류 문제**에서 사용된다. 이진 분류에서는 하나의 출력으로 충분하기 때문이다.
  - 예를 들어 스팸 메시지인지, 아닌지 예측하는 문제에서는 스팸 OK, 스팸 X로만 분류하면 되기 때문이다.
  - Binary Cross-entropy(BCE)와 같은 손실 함수를 사용하여 Loss를 계산한다.

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i))$$

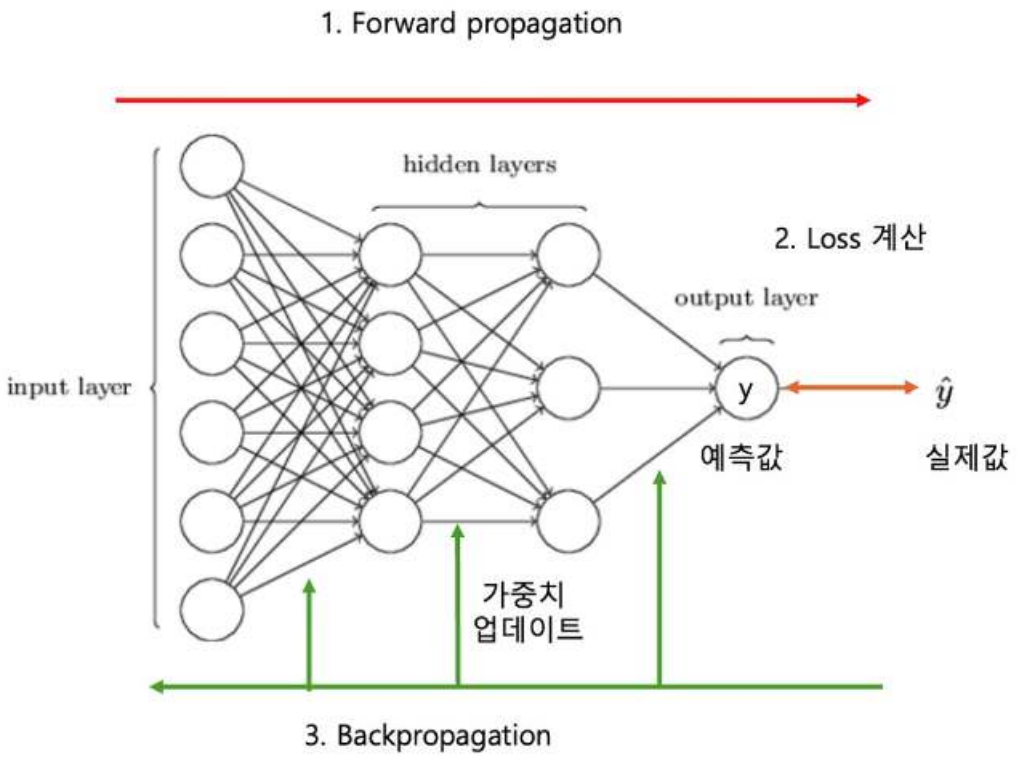
- **여러 개의 출력**: **다중 클래스 분류나 회귀 문제 등**은 여러 개의 특징들이 필요하다.
  - 예를 들면 동물을 분류해야 한다고 생각하면, 강아지, 고양이, 사자 등 여러개의 클래스들 중 예측해야 할 것이다.
  - 클래스 분류 문제에서는 각 클래스에 대한 확률 값을 출력하여 가장 가능성 있는 클래스를 선택하는 방식을 사용한다.
  - Categorical Cross-entropy(CCE)와 같은 손실 함수를 사용하여 Loss를 계산한다.

$$\boldsymbol{CCE = -\sum_{i=1}^{C} y_i \log \hat{y}_i}$$

https://sjh9708.tistory.com/225

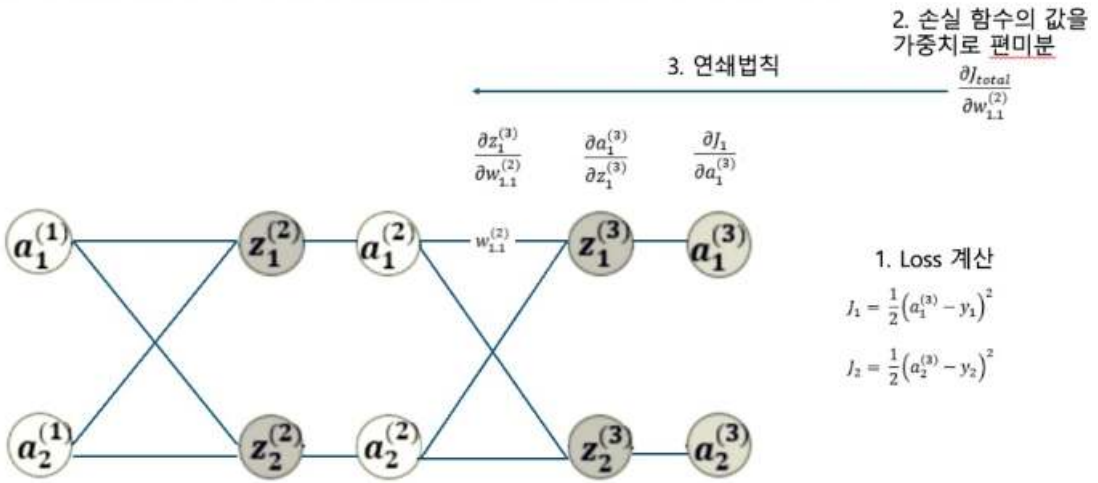
Appendix - MLP – How to train?

3. Backpropagation (역전파)



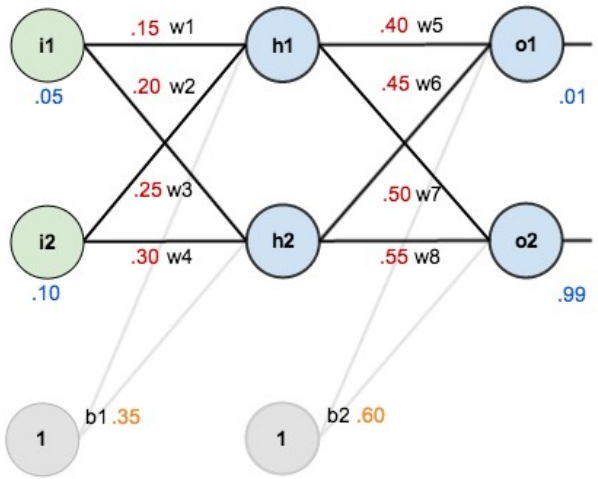
역전파 알고리즘

출력층에서 입력층으로 거꾸로 진행하면서 각각의 Hidden Layer의 각각의 가중치들을 업데이트 시켜주는 작업을 진행한다.



https://sjh9708.tistory.com/225

Appendix - MLP – How to train?



① Feed forward

오차역전파를 위해서는 우선 순방향으로 결과 값을 먼저 계산한 후에 그 **오차(Error)**를 구해야 합니다.
하나씩 순서대로 계산해보겠습니다.

$$net_{h_1} = i_1 * w_1 + i_2 * w_2 + b_1 * 1 = 0.05 * 0.15 + 0.1 * 0.2 + 0.35 * 1 = 0.3775$$

$$out_{h_1} = \frac{1}{1 + e^{-net_{h_1}}} = 0.593269992$$

같은 방법으로 계산하면 $out_{h_2} = 0.596884378$,
따라서 $out_{o_1} = 0.75136507, out_{o_2} = 0.772928465$가 됩니다.

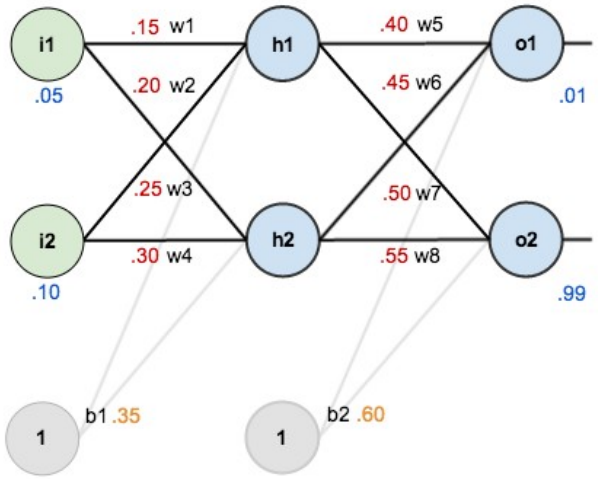
② Error 계산

이제 위에서 계산한 결과 값을 이용해서 오차를 계산해보겠습니다.

$E = \sum \frac{1}{2}(target - output)^2$이므로
$E_{total} = E_{o_1} + E_{o_2} = \frac{1}{2}(0.01 - 0.75136507)^2 + \frac{1}{2}(0.99 - 0.772928465)^2$
$= 0.274811083 + 0.023560026 = 0.298371109$

https://nanunzoey.tistory.com/entry/ 딥러닝 - 오차역전파 Back-propagation- 직접 - 계산하기

Appendix - MLP – How to train?

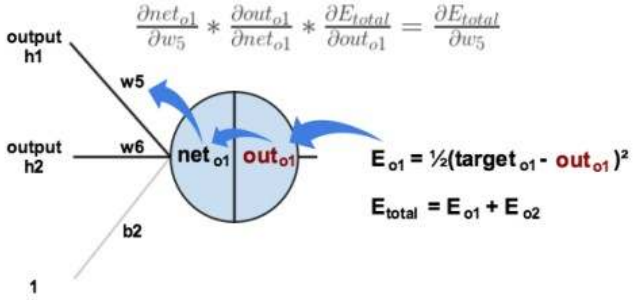


③ Back-propagation

오차역전파는 신경망의 역방향으로 **오차의 기울기(Gradient)**를 계산해 **가중치를 업데이트**하는 과정입
따라서 먼저 신경망의 최종 output으로부터 전체 오차의 그래디언트를 계산해보겠습니다.

③-1 마지막 layer에서 오차 gradient 계산

$$\frac{\partial net_{o1}}{\partial w_5} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial E_{total}}{\partial out_{o1}} = \frac{\partial E_{total}}{\partial w_5}$$



https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

가중치 w_5에 대한 전체 오차의 기울기 $\frac{\partial E_{total}}{\partial w_5}$는 Chain rule에 의해 다음과 같이 계산이 가능합니다.

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial net_{o_1}} \times \frac{\partial net_{o_1}}{\partial w_5}$$

위 세 가지 항(term)을 차례대로 구해보겠습니다.

- $\frac{\partial E_{total}}{\partial out_{o_1}} = 2 \times \frac{1}{2} target_{o_1} - out_{o_1}{}^{2-1} \times (-1)$

$$= -target_{o_1} - output_{o_1} = -(0.01 - 0.75136507) = 0.74136507$$

- $out_{o_1} = \frac{1}{1+e^{-net_{o1}}}$이므로

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = out_{o_1}(1 - out_{o_1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$

- $net_{o_1} = w_5 * out_{h_1} + w_6 * out_{h_2} + b_2 * 1$이므로

$$\frac{\partial net_{o_1}}{\partial w_5} = out_{h_1} = 0.593269992$$

따라서,

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 \times 0.186815602 \times 0.593269992 = 0.082167041$$

③-2 해당 가중치 업데이트

이제 위에서 계산한 w_5에 대한 오차 E_{total}의 편미분, 즉 기울기 값을 이용해 해당 가중치를 업데이트할 수 있습니다. 이를 수식으로 나타내면 다음과 같습니다.

$$w_5^+ = w_5 - \eta \times \frac{\partial E_{total}}{\partial w_5}$$

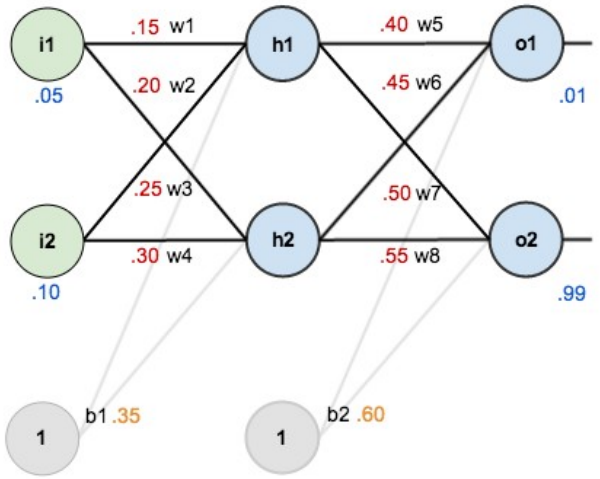
이때 η는 하이퍼파라미터로서 학습률(Learning rate)을 의미하며, 임의로 0.5로 설정해보겠습니다.
따라서 앞서 구한 값들을 이용해 계산해보면, $w_5^+ = 0.4 - 0.5 \times 0.082167041 = 0.35891648$이 됩니다.
기존 0.4에서 조금 더 작은 값이 되었습니다.

마찬가지로 나머지 가중치도 계산하면, 다음과 같이 업데이트할 수 있습니다.

- $w_6^+ = 0.45 \rightarrow 0.408666186$
- $w_7^+ = 0.5 \rightarrow 0.511301270$
- $w_8^+ = 0.55 \rightarrow 0.561370121$

https://nanunzoey.tistory.com/entry/ 딥러닝 - 오차역전파 Back-propagation- 직접 - 계산하기

Appendix - MLP – How to train?



③-3 은닉층(Hidden Layer)에서 오차 gradient 계산

아직 끝난 게 아닙니다! 바로 은닉층의 가중치 업데이트가 남아있죠.
이를 계산하기 위해서는 역시 은닉층에서 오차의 기울기를 먼저 구해야 합니다. 그 과정은 위 ①에서와 거의 유사하지만, 몇 가지 유념할 부분이 있습니다.

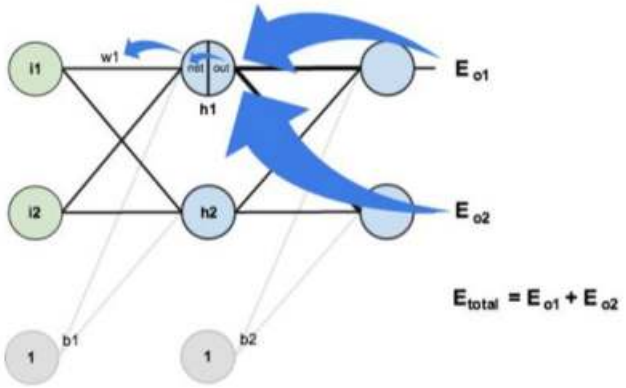
우선, 이번에는 w_1에 대한 오차의 기울기를 구해보겠습니다. 역시 Chain rule에 따라 다음과 같이 풀어서 계산이 가능합니다.

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h_1}} \times \frac{\partial out_{h_1}}{\partial net_{h_1}} \times \frac{\partial net_{h_1}}{\partial w_1}$$

마찬가지로 세 가지 항을 차례대로 구한 뒤 서로 곱해보겠습니다.

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

첫 번째 항을 구할 때 주의할 점은 해당 노드에서 오차 E_{total}가 여러 노드들의 오차가 더해진 값이라는 사실입니다.
즉, $\frac{\partial E_{total}}{\partial out_{h_1}}$는 $\frac{\partial E_{o_1}}{\partial out_{h_1}}$와 $\frac{\partial E_{o_2}}{\partial out_{h_1}}$를 각각 더해서 계산해야 합니다.

- $\frac{\partial E_{o_1}}{\partial out_{h_1}} = \frac{\partial E_{o_1}}{\partial net_{o_1}} \times \frac{\partial net_{o_1}}{\partial out_{h_1}} = (\frac{\partial E_{o_1}}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial net_{o_1}}) \times w_5$

$= (0.74136507 \times 0.186815602) \times 0.4 = 0.055399425$

이때 또 한 가지 주의할 것은 아직 전체 신경망에 대한 back-prop이 끝나지 않았기 때문에 ②에서 계산한 새로운 w_5^+가 아닌 기존 w_5를 이용해야 한다는 것입니다.

- $\frac{\partial E_{o_2}}{\partial out_{h_1}} = -0.019049119$이므로

$\frac{\partial E_{total}}{\partial out_{h_1}} = 0.055399425 + (-0.019049119) = 0.036350306$

나머지 두 항은 이전에 계산한 값들로 비교적 빠르게 계산이 가능합니다.

- $out_{h_1} = \frac{1}{1+e^{-net_{h_1}}}$이므로

$\frac{\partial out_{h_1}}{\partial net_{h_1}} = out_{h_1}(1 - out_{h_1}) = 0.59326999(1 - 0.59326999) = 0.241300709$

- $net_{h_1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$이므로 $\frac{\partial net_{h_1}}{\partial w_1} = i_1 = 0.05$

따라서,

$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 \times 0.241300709 \times 0.05 = 0.000438568$

https://nanunzoey.tistory.com/entry/ 딥러닝 - 오차역전파 Back-propagation- 직접 - 계산하기

Appendix - Constrained Policy Optimization paper review - again

Large or continuous MDP → solving exact optimal policy is intractable

↓ iteratively update policy by maximizing J(π) over a local neighborhood of the most recent iterate π_k

CMDP

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J(\pi)$$
$$\text{s.t. } J_{C_i}(\pi) \leq d_i \quad i = 1, ..., m$$
$$D(\pi, \pi_k) \leq \delta.$$

Difficult to implement in practice
∵ requires evaluation of the constraint functions to determine π is feasible.

↓ replace the objective and constraints with surrogate functions

CPO

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} [A^{\pi_k}(s,a)]$$
$$\text{s.t. } J_{C_i}(\pi_k) + \frac{1}{1-\gamma} \mathop{\mathrm{E}}_{\substack{s \sim d^{\pi_k} \\ a \sim \pi}} \left[A^{\pi_k}_{C_i}(s,a)\right] \leq d_i \quad \forall i$$
$$\bar{D}_{KL}(\pi || \pi_k) \leq \delta. \tag{10}$$

Policies with high-dimensional parameter spaces like neural networks
→ (10) is impractical to solve directly because of the computational cost

↓ For small step sizes δ
objective and cost constraints are well-approximated by linearizing around π_k

$$\theta_{k+1} = \arg\max_{\theta} \; g^T(\theta - \theta_k)$$
$$\text{s.t.} \quad c_i + b_i^T(\theta - \theta_k) \leq 0 \quad i = 1, ..., m$$
$$\frac{1}{2}(\theta - \theta_k)^T H (\theta - \theta_k) \leq \delta.$$
(11)

Appendix - Constrained Policy Optimization paper review - again

$$\begin{aligned}\theta_{k+1} = \arg\max_{\theta} \quad & g^T(\theta - \theta_k) \\ \text{s.t.} \quad & c_i + b_i^T(\theta - \theta_k) \leq 0 \quad i = 1, ..., m \\ & \frac{1}{2}(\theta - \theta_k)^T H(\theta - \theta_k) \leq \delta.\end{aligned} \tag{11}$$

Dual problem

$$\max_{\substack{\lambda \geq 0 \\ \nu \succeq 0}} \frac{-1}{2\lambda}\left(g^T H^{-1} g - 2r^T\nu + \nu^T S\nu\right) + \nu^T c - \frac{\lambda\delta}{2}, \tag{12}$$

Approximation error →
add recovery / backtracking linear search step

Algorithm 1 Constrained Policy Optimization

Input: Initial policy $\pi_0 \in \Pi_\theta$ tolerance α
for $k = 0, 1, 2, ...$ **do**
  Sample a set of trajectories $\mathcal{D} = \{\tau\} \sim \pi_k = \pi(\theta_k)$
  Form sample estimates $\hat{g}, \hat{b}, \hat{H}, \hat{c}$ with $\mathcal{D}$
  if approximate CPO is feasible **then**
    Solve dual problem (12) for λ_k^*, ν_k^*
    Compute policy proposal θ^* with (13)
  else
    Compute recovery policy proposal θ^* with (14)
  end if
  Obtain θ_{k+1} by backtracking linesearch to enforce satisfaction of sample estimates of constraints in (10)
end for